For the solutions to exercises 7 and 8, please see the Jupyter notebooks available at *https://github.com/ageron/handson-ml*.

# Chapter 7: Ensemble Learning and Random Forests

1. If you have trained five different models and they all achieve 95% precision, you can try combining them into a voting ensemble, which will often give you even better results. It works better if the models are very different (e.g., an SVM classifier, a Decision Tree classifier, a Logistic Regression classifier, and so on). It is even better if they are trained on different training instances (that's the whole point of bagging and pasting ensembles), but if not it will still work as long as the models are very different.
2. A hard voting classifier just counts the votes of each classifier in the ensemble and picks the class that gets the most votes. A soft voting classifier computes the average estimated class probability for each class and picks the class with the highest probability. This gives high-confidence votes more weight and often performs better, but it works only if every classifier is able to estimate class probabilities (e.g., for the SVM classifiers in Scikit-Learn you must set `probability=True`).
3. It is quite possible to speed up training of a bagging ensemble by distributing it across multiple servers, since each predictor in the ensemble is independent of the others. The same goes for pasting ensembles and Random Forests, for the same reason. However, each predictor in a boosting ensemble is built based on the previous predictor, so training is necessarily sequential, and you will not gain anything by distributing training across multiple servers. Regarding stacking ensembles, all the predictors in a given layer are independent of each other, so they can be trained in parallel on multiple servers. However, the predictors in one layer can only be trained after the predictors in the previous layer have all been trained.
4. With out-of-bag evaluation, each predictor in a bagging ensemble is evaluated using instances that it was not trained on (they were held out). This makes it possible to have a fairly unbiased evaluation of the ensemble without the need for an additional validation set. Thus, you have more instances available for training, and your ensemble can perform slightly better.
5. When you are growing a tree in a Random Forest, only a random subset of the features is considered for splitting at each node. This is true as well for Extra-Trees, but they go one step further: rather than searching for the best possible thresholds, like regular Decision Trees do, they use random thresholds for each feature. This extra randomness acts like a form of regularization: if a Random Forest overfits the training data, Extra-Trees might perform better. Moreover, since Extra-Trees don't search for the best possible thresholds, they are much

faster to train than Random Forests. However, they are neither faster nor slower than Random Forests when making predictions.

6. If your AdaBoost ensemble underfits the training data, you can try increasing the number of estimators or reducing the regularization hyperparameters of the base estimator. You may also try slightly increasing the learning rate.
7. If your Gradient Boosting ensemble overfits the training set, you should try decreasing the learning rate. You could also use early stopping to find the right number of predictors (you probably have too many).

For the solutions to exercises 8 and 9, please see the Jupyter notebooks available at *https://github.com/ageron/handson-ml*.

# Chapter 8: Dimensionality Reduction

1. Motivations and drawbacks:
   - The main motivations for dimensionality reduction are:
     - To speed up a subsequent training algorithm (in some cases it may even remove noise and redundant features, making the training algorithm perform better).
     - To visualize the data and gain insights on the most important features.
     - Simply to save space (compression).
   - The main drawbacks are:
     - Some information is lost, possibly degrading the performance of subsequent training algorithms.
     - It can be computationally intensive.
     - It adds some complexity to your Machine Learning pipelines.
     - Transformed features are often hard to interpret.
2. The curse of dimensionality refers to the fact that many problems that do not exist in low-dimensional space arise in high-dimensional space. In Machine Learning, one common manifestation is the fact that randomly sampled high-dimensional vectors are generally very sparse, increasing the risk of overfitting and making it very difficult to identify patterns in the data without having plenty of training data.
3. Once a dataset's dimensionality has been reduced using one of the algorithms we discussed, it is almost always impossible to perfectly reverse the operation, because some information gets lost during dimensionality reduction. Moreover, while some algorithms (such as PCA) have a simple reverse transformation pro-

cedure that can reconstruct a dataset relatively similar to the original, other algorithms (such as T-SNE) do not.

4. PCA can be used to significantly reduce the dimensionality of most datasets, even if they are highly nonlinear, because it can at least get rid of useless dimensions. However, if there are no useless dimensions—for example, the Swiss roll—then reducing dimensionality with PCA will lose too much information. You want to unroll the Swiss roll, not squash it.
5. That's a trick question: it depends on the dataset. Let's look at two extreme examples. First, suppose the dataset is composed of points that are almost perfectly aligned. In this case, PCA can reduce the dataset down to just one dimension while still preserving 95% of the variance. Now imagine that the dataset is composed of perfectly random points, scattered all around the 1,000 dimensions. In this case roughly 950 dimensions are required to preserve 95% of the variance. So the answer is, it depends on the dataset, and it could be any number between 1 and 950. Plotting the explained variance as a function of the number of dimensions is one way to get a rough idea of the dataset's intrinsic dimensionality.
6. Regular PCA is the default, but it works only if the dataset fits in memory. Incremental PCA is useful for large datasets that don't fit in memory, but it is slower than regular PCA, so if the dataset fits in memory you should prefer regular PCA. Incremental PCA is also useful for online tasks, when you need to apply PCA on the fly, every time a new instance arrives. Randomized PCA is useful when you want to considerably reduce dimensionality and the dataset fits in memory; in this case, it is much faster than regular PCA. Finally, Kernel PCA is useful for nonlinear datasets.
7. Intuitively, a dimensionality reduction algorithm performs well if it eliminates a lot of dimensions from the dataset without losing too much information. One way to measure this is to apply the reverse transformation and measure the reconstruction error. However, not all dimensionality reduction algorithms provide a reverse transformation. Alternatively, if you are using dimensionality reduction as a preprocessing step before another Machine Learning algorithm (e.g., a Random Forest classifier), then you can simply measure the performance of that second algorithm; if dimensionality reduction did not lose too much information, then the algorithm should perform just as well as when using the original dataset.
8. It can absolutely make sense to chain two different dimensionality reduction algorithms. A common example is using PCA to quickly get rid of a large number of useless dimensions, then applying another much slower dimensionality reduction algorithm, such as LLE. This two-step approach will likely yield the same performance as using LLE only, but in a fraction of the time.

For the solutions to exercises 9 and 10, please see the Jupyter notebooks available at *https://github.com/ageron/handson-ml*.

# Chapter 9: Up and Running with TensorFlow

1. Main benefits and drawbacks of creating a computation graph rather than directly executing the computations:
   - Main benefits:
     - TensorFlow can automatically compute the gradients for you (using reverse-mode autodiff).
     - TensorFlow can take care of running the operations in parallel in different threads.
     - It makes it easier to run the same model across different devices.
     - It simplifies introspection—for example, to view the model in TensorBoard.
   - Main drawbacks:
     - It makes the learning curve steeper.
     - It makes step-by-step debugging harder.
2. Yes, the statement `a_val = a.eval(session=sess)` is indeed equivalent to `a_val = sess.run(a)`.
3. No, the statement `a_val, b_val = a.eval(session=sess), b.eval(session=sess)` is not equivalent to `a_val, b_val = sess.run([a, b])`. Indeed, the first statement runs the graph twice (once to compute `a`, once to compute `b`), while the second statement runs the graph only once. If any of these operations (or the ops they depend on) have side effects (e.g., a variable is modified, an item is inserted in a queue, or a reader reads a file), then the effects will be different. If they don't have side effects, both statements will return the same result, but the second statement will be faster than the first.
4. No, you cannot run two graphs in the same session. You would have to merge the graphs into a single graph first.
5. In local TensorFlow, sessions manage variable values, so if you create a graph `g` containing a variable `w`, then start two threads and open a local session in each thread, both using the same graph `g`, then each session will have its own copy of the variable `w`. However, in distributed TensorFlow, variable values are stored in containers managed by the cluster, so if both sessions connect to the same cluster and use the same container, then they will share the same variable value for `w`.
6. A variable is initialized when you call its initializer, and it is destroyed when the session ends. In distributed TensorFlow, variables live in containers on the clus-

ter, so closing a session will not destroy the variable. To destroy a variable, you need to clear its container.

7. Variables and placeholders are extremely different, but beginners often confuse them:

   - A variable is an operation that holds a value. If you run the variable, it returns that value. Before you can run it, you need to initialize it. You can change the variable's value (for example, by using an assignment operation). It is stateful: the variable keeps the same value upon successive runs of the graph. It is typically used to hold model parameters but also for other purposes (e.g., to count the global training step).
   - Placeholders technically don't do much: they just hold information about the type and shape of the tensor they represent, but they have no value. In fact, if you try to evaluate an operation that depends on a placeholder, you must feed TensorFlow the value of the placeholder (using the `feed_dict` argument) or else you will get an exception. Placeholders are typically used to feed training or test data to TensorFlow during the execution phase. They are also useful to pass a value to an assignment node, to change the value of a variable (e.g., model weights).

8. If you run the graph to evaluate an operation that depends on a placeholder but you don't feed its value, you get an exception. If the operation does not depend on the placeholder, then no exception is raised.
9. When you run a graph, you can feed the output value of any operation, not just the value of placeholders. In practice, however, this is rather rare (it can be useful, for example, when you are caching the output of frozen layers; see Chapter 11).
10. You can specify a variable's initial value when constructing the graph, and it will be initialized later when you run the variable's initializer during the execution phase. If you want to change that variable's value to anything you want during the execution phase, then the simplest option is to create an assignment node (during the graph construction phase) using the `tf.assign()` function, passing the variable and a placeholder as parameters. During the execution phase, you can run the assignment operation and feed the variable's new value using the placeholder.

```
import tensorflow as tf

x = tf.Variable(tf.random_uniform(shape=(), minval=0.0, maxval=1.0))
x_new_val = tf.placeholder(shape=(), dtype=tf.float32)
x_assign = tf.assign(x, x_new_val)

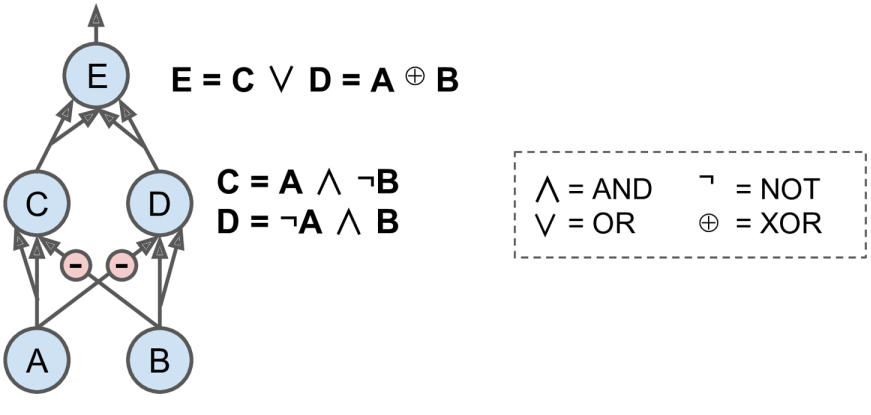
with tf.Session():
    x.initializer.run() # random number is sampled *now*
    print(x.eval()) # 0.646157 (some random number)
```

```
x_assign.eval(feed_dict={x_new_val: 5.0})
print(x.eval()) # 5.0
```

11. Reverse-mode autodiff (implemented by TensorFlow) needs to traverse the graph only twice in order to compute the gradients of the cost function with regards to any number of variables. On the other hand, forward-mode autodiff would need to run once for each variable (so 10 times if we want the gradients with regards to 10 different variables). As for symbolic differentiation, it would build a different graph to compute the gradients, so it would not traverse the original graph at all (except when building the new gradients graph). A highly optimized symbolic differentiation system could potentially run the new gradients graph only once to compute the gradients with regards to all variables, but that new graph may be horribly complex and inefficient compared to the original graph.
12. See the Jupyter notebooks available at *https://github.com/ageron/handson-ml*.

# Chapter 10: Introduction to Artificial Neural Networks

1. Here is a neural network based on the original artificial neurons that computes $A \oplus B$ (where $\oplus$ represents the exclusive OR), using the fact that $A \oplus B = (A \wedge \neg B) \vee (\neg A \wedge B)$. There are other solutions—for example, using the fact that $A \oplus B = (A \vee B) \wedge \neg(A \wedge B)$, or the fact that $A \oplus B = (A \vee B) \wedge (\neg A \vee \wedge B)$, and so on.




2. A classical Perceptron will converge only if the dataset is linearly separable, and it won't be able to estimate class probabilities. In contrast, a Logistic Regression classifier will converge to a good solution even if the dataset is not linearly separable, and it will output class probabilities. If you change the Perceptron's activation function to the logistic activation function (or the softmax activation function if there are multiple neurons), and if you train it using Gradient Descent

(or some other optimization algorithm minimizing the cost function, typically cross entropy), then it becomes equivalent to a Logistic Regression classifier.

3. The logistic activation function was a key ingredient in training the first MLPs because its derivative is always nonzero, so Gradient Descent can always roll down the slope. When the activation function is a step function, Gradient Descent cannot move, as there is no slope at all.
4. The step function, the logistic function, the hyperbolic tangent, the rectified linear unit (see Figure 10-8). See Chapter 11 for other examples, such as ELU and variants of the ReLU.
5. Considering the MLP described in the question: suppose you have an MLP composed of one input layer with 10 passthrough neurons, followed by one hidden layer with 50 artificial neurons, and finally one output layer with 3 artificial neurons. All artificial neurons use the ReLU activation function.

   - The shape of the input matrix $\mathbf{X}$ is $m \times 10$, where $m$ represents the training batch size.
   - The shape of the hidden layer's weight vector $\mathbf{W}_h$ is $10 \times 50$ and the length of its bias vector $\mathbf{b}_h$ is 50.
   - The shape of the output layer's weight vector $\mathbf{W}_o$ is $50 \times 3$, and the length of its bias vector $\mathbf{b}_o$ is 3.
   - The shape of the network's output matrix $\mathbf{Y}$ is $m \times 3$.
   - $\mathbf{Y} = \text{ReLU}(\text{ReLU}(\mathbf{X} \cdot \mathbf{W}_h + \mathbf{b}_h) \cdot \mathbf{W}_o + \mathbf{b}_o)$. Recall that the ReLU function just sets every negative number in the matrix to zero. Also note that when you are adding a bias vector to a matrix, it is added to every single row in the matrix, which is called *broadcasting*.

6. To classify email into spam or ham, you just need one neuron in the output layer of a neural network—for example, indicating the probability that the email is spam. You would typically use the logistic activation function in the output layer when estimating a probability. If instead you want to tackle MNIST, you need 10 neurons in the output layer, and you must replace the logistic function with the softmax activation function, which can handle multiple classes, outputting one probability per class. Now, if you want your neural network to predict housing prices like in Chapter 2, then you need one output neuron, using no activation function at all in the output layer.[4]

4 When the values to predict can vary by many orders of magnitude, then you may want to predict the logarithm of the target value rather than the target value directly. Simply computing the exponential of the neural network's output will give you the estimated value (since $\exp(\log v) = v$).

7. Backpropagation is a technique used to train artificial neural networks. It first computes the gradients of the cost function with regards to every model parameter (all the weights and biases), and then it performs a Gradient Descent step using these gradients. This backpropagation step is typically performed thousands or millions of times, using many training batches, until the model parameters converge to values that (hopefully) minimize the cost function. To compute the gradients, backpropagation uses reverse-mode autodiff (although it wasn't called that when backpropagation was invented, and it has been reinvented several times). Reverse-mode autodiff performs a forward pass through a computation graph, computing every node's value for the current training batch, and then it performs a reverse pass, computing all the gradients at once (see Appendix D for more details). So what's the difference? Well, backpropagation refers to the whole process of training an artificial neural network using multiple backpropagation steps, each of which computes gradients and uses them to perform a Gradient Descent step. In contrast, reverse-mode autodiff is a simply a technique to compute gradients efficiently, and it happens to be used by backpropagation.
8. Here is a list of all the hyperparameters you can tweak in a basic MLP: the number of hidden layers, the number of neurons in each hidden layer, and the activation function used in each hidden layer and in the output layer.[5] In general, the ReLU activation function (or one of its variants; see Chapter 11) is a good default for the hidden layers. For the output layer, in general you will want the logistic activation function for binary classification, the softmax activation function for multiclass classification, or no activation function for regression.

   If the MLP overfits the training data, you can try reducing the number of hidden layers and reducing the number of neurons per hidden layer.
9. See the Jupyter notebooks available at *https://github.com/ageron/handson-ml*.

# Chapter 11: Training Deep Neural Nets

1. No, all weights should be sampled independently; they should not all have the same initial value. One important goal of sampling weights randomly is to break symmetries: if all the weights have the same initial value, even if that value is not zero, then symmetry is not broken (i.e., all neurons in a given layer are equivalent), and backpropagation will be unable to break it. Concretely, this means that

5 In Chapter 11 we discuss many techniques that introduce additional hyperparameters: type of weight initialization, activation function hyperparameters (e.g., amount of leak in leaky ReLU), Gradient Clipping threshold, type of optimizer and its hyperparameters (e.g., the momentum hyperparameter when using a `MomentumOptimizer`), type of regularization for each layer, and the regularization hyperparameters (e.g., dropout rate when using dropout) and so on.

all the neurons in any given layer will always have the same weights. It's like having just one neuron per layer, and much slower. It is virtually impossible for such a configuration to converge to a good solution.

2. It is perfectly fine to initialize the bias terms to zero. Some people like to initialize them just like weights, and that's okay too; it does not make much difference.
3. A few advantages of the ELU function over the ReLU function are:

   - It can take on negative values, so the average output of the neurons in any given layer is typically closer to 0 than when using the ReLU activation function (which never outputs negative values). This helps alleviate the vanishing gradients problem.
   - It always has a nonzero derivative, which avoids the dying units issue that can affect ReLU units.
   - It is smooth everywhere, whereas the ReLU's slope abruptly jumps from 0 to 1 at $z = 0$. Such an abrupt change can slow down Gradient Descent because it will bounce around $z = 0$.

4. The ELU activation function is a good default. If you need the neural network to be as fast as possible, you can use one of the leaky ReLU variants instead (e.g., a simple leaky ReLU using the default hyperparameter value). The simplicity of the ReLU activation function makes it many people's preferred option, despite the fact that they are generally outperformed by the ELU and leaky ReLU. However, the ReLU activation function's capability of outputting precisely zero can be useful in some cases (e.g., see Chapter 15). The hyperbolic tangent (tanh) can be useful in the output layer if you need to output a number between –1 and 1, but nowadays it is not used much in hidden layers. The logistic activation function is also useful in the output layer when you need to estimate a probability (e.g., for binary classification), but it is also rarely used in hidden layers (there are exceptions—for example, for the coding layer of variational autoencoders; see Chapter 15). Finally, the softmax activation function is useful in the output layer to output probabilities for mutually exclusive classes, but other than that it is rarely (if ever) used in hidden layers.
5. If you set the `momentum` hyperparameter too close to 1 (e.g., 0.99999) when using a `MomentumOptimizer`, then the algorithm will likely pick up a lot of speed, hopefully roughly toward the global minimum, but then it will shoot right past the minimum, due to its momentum. Then it will slow down and come back, accelerate again, overshoot again, and so on. It may oscillate this way many times before converging, so overall it will take much longer to converge than with a smaller `momentum` value.
6. One way to produce a sparse model (i.e., with most weights equal to zero) is to train the model normally, then zero out tiny weights. For more sparsity, you can

apply $\ell_1$ regularization during training, which pushes the optimizer toward sparsity. A third option is to combine $\ell_1$ regularization with *dual averaging*, using TensorFlow's `FTRLOptimizer` class.

7. Yes, dropout does slow down training, in general roughly by a factor of two. However, it has no impact on inference since it is only turned on during training.

For the solutions to exercises 8, 9, and 10, please see the Jupyter notebooks available at *https://github.com/ageron/handson-ml*.

# Chapter 12: Distributing TensorFlow Across Devices and Servers

1. When a TensorFlow process starts, it grabs all the available memory on all GPU devices that are visible to it, so if you get a `CUDA_ERROR_OUT_OF_MEMORY` when starting your TensorFlow program, it probably means that other processes are running that have already grabbed all the memory on at least one visible GPU device (most likely it is another TensorFlow process). To fix this problem, a trivial solution is to stop the other processes and try again. However, if you need all processes to run simultaneously, a simple option is to dedicate different devices to each process, by setting the `CUDA_VISIBLE_DEVICES` environment variable appropriately for each device. Another option is to configure TensorFlow to grab only part of the GPU memory, instead of all of it, by creating a `ConfigProto`, setting its `gpu_options.per_process_gpu_memory_fraction` to the proportion of the total memory that it should grab (e.g., 0.4), and using this `ConfigProto` when opening a session. The last option is to tell TensorFlow to grab memory only when it needs it by setting the `gpu_options.allow_growth` to `True`. However, this last option is usually not recommended because any memory that TensorFlow grabs is never released, and it is harder to guarantee a repeatable behavior (there may be race conditions depending on which processes start first, how much memory they need during training, and so on).
2. By pinning an operation on a device, you are telling TensorFlow that this is where you would like this operation to be placed. However, some constraints may prevent TensorFlow from honoring your request. For example, the operation may have no implementation (called a *kernel*) for that particular type of device. In this case, TensorFlow will raise an exception by default, but you can configure it to fall back to the CPU instead (this is called *soft placement*). Another example is an operation that can modify a variable; this operation and the variable need to be collocated. So the difference between pinning an operation and placing an operation is that pinning is what you ask TensorFlow ("Please place this operation on GPU #1") while placement is what TensorFlow actually ends up doing ("Sorry, falling back to the CPU").

3. If you are running on a GPU-enabled TensorFlow installation, and you just use the default placement, then if all operations have a GPU kernel (i.e., a GPU implementation), yes, they will all be placed on the first GPU. However, if one or more operations do not have a GPU kernel, then by default TensorFlow will raise an exception. If you configure TensorFlow to fall back to the CPU instead (soft placement), then all operations will be placed on the first GPU except the ones without a GPU kernel and all the operations that must be collocated with them (see the answer to the previous exercise).
4. Yes, if you pin a variable to `/gpu:0`, it can be used by operations placed on `/gpu:1`. TensorFlow will automatically take care of adding the appropriate operations to transfer the variable's value across devices. The same goes for devices located on different servers (as long as they are part of the same cluster).
5. Yes, two operations placed on the same device can run in parallel: TensorFlow automatically takes care of running operations in parallel (on different CPU cores or different GPU threads), as long as no operation depends on another operation's output. Moreover, you can start multiple sessions in parallel threads (or processes), and evaluate operations in each thread. Since sessions are independent, TensorFlow will be able to evaluate any operation from one session in parallel with any operation from another session.
6. Control dependencies are used when you want to postpone the evaluation of an operation X until after some other operations are run, even though these operations are not required to compute X. This is useful in particular when X would occupy a lot of memory and you only need it later in the computation graph, or if X uses up a lot of I/O (for example, it requires a large variable value located on a different device or server) and you don't want it to run at the same time as other I/O-hungry operations, to avoid saturating the bandwidth.
7. You're in luck! In distributed TensorFlow, the variable values live in containers managed by the cluster, so even if you close the session and exit the client program, the model parameters are still alive and well on the cluster. You simply need to open a new session to the cluster and save the model (make sure you don't call the variable initializers or restore a previous model, as this would destroy your precious new model!).

For the solutions to exercises 8, 9, and 10, please see the Jupyter notebooks available at *https://github.com/ageron/handson-ml*.

# Chapter 13: Convolutional Neural Networks

1. These are the main advantages of a CNN over a fully connected DNN for image classification:

- Because consecutive layers are only partially connected and because it heavily reuses its weights, a CNN has many fewer parameters than a fully connected DNN, which makes it much faster to train, reduces the risk of overfitting, and requires much less training data.
- When a CNN has learned a kernel that can detect a particular feature, it can detect that feature anywhere on the image. In contrast, when a DNN learns a feature in one location, it can detect it only in that particular location. Since images typically have very repetitive features, CNNs are able to generalize much better than DNNs for image processing tasks such as classification, using fewer training examples.
- Finally, a DNN has no prior knowledge of how pixels are organized; it does not know that nearby pixels are close. A CNN's architecture embeds this prior knowledge. Lower layers typically identify features in small areas of the images, while higher layers combine the lower-level features into larger features. This works well with most natural images, giving CNNs a decisive head start compared to DNNs.

2. Let's compute how many parameters the CNN has. Since its first convolutional layer has 3 × 3 kernels, and the input has three channels (red, green, and blue), then each feature map has 3 × 3 × 3 weights, plus a bias term. That's 28 parameters per feature map. Since this first convolutional layer has 100 feature maps, it has a total of 2,800 parameters. The second convolutional layer has 3 × 3 kernels, and its input is the set of 100 feature maps of the previous layer, so each feature map has 3 × 3 × 100 = 900 weights, plus a bias term. Since it has 200 feature maps, this layer has 901 × 200 = 180,200 parameters. Finally, the third and last convolutional layer also has 3 × 3 kernels, and its input is the set of 200 feature maps of the previous layers, so each feature map has 3 × 3 × 200 = 1,800 weights, plus a bias term. Since it has 400 feature maps, this layer has a total of 1,801 × 400 = 720,400 parameters. All in all, the CNN has 2,800 + 180,200 + 720,400 = 903,400 parameters.

   Now let's compute how much RAM this neural network will require (at least) when making a prediction for a single instance. First let's compute the feature map size for each layer. Since we are using a stride of 2 and SAME padding, the horizontal and vertical size of the feature maps are divided by 2 at each layer (rounding up if necessary), so as the input channels are 200 × 300 pixels, the first layer's feature maps are 100 × 150, the second layer's feature maps are 50 × 75, and the third layer's feature maps are 25 × 38. Since 32 bits is 4 bytes and the first convolutional layer has 100 feature maps, this first layer takes up 4 x 100 × 150 × 100 = 6 million bytes (about 5.7 MB, considering that 1 MB = 1,024 KB and 1 KB = 1,024 bytes). The second layer takes up 4 × 50 × 75 × 200 = 3 million bytes (about 2.9 MB). Finally, the third layer takes up 4 × 25 × 38 × 400 = 1,520,000

bytes (about 1.4 MB). However, once a layer has been computed, the memory occupied by the previous layer can be released, so if everything is well optimized, only 6 + 3 = 9 million bytes (about 8.6 MB) of RAM will be required (when the second layer has just been computed, but the memory occupied by the first layer is not released yet). But wait, you also need to add the memory occupied by the CNN's parameters. We computed earlier that it has 903,400 parameters, each using up 4 bytes, so this adds 3,613,600 bytes (about 3.4 MB). The total RAM required is (at least) 12,613,600 bytes (about 12.0 MB).

Lastly, let's compute the minimum amount of RAM required when training the CNN on a mini-batch of 50 images. During training TensorFlow uses backpropagation, which requires keeping all values computed during the forward pass until the reverse pass begins. So we must compute the total RAM required by all layers for a single instance and multiply that by 50! At that point let's start counting in megabytes rather than bytes. We computed before that the three layers require respectively 5.7, 2.9, and 1.4 MB for each instance. That's a total of 10.0 MB per instance. So for 50 instances the total RAM is 500 MB. Add to that the RAM required by the input images, which is $50 \times 4 \times 200 \times 300 \times 3 = 36$ million bytes (about 34.3 MB), plus the RAM required for the model parameters, which is about 3.4 MB (computed earlier), plus some RAM for the gradients (we will neglect them since they can be released gradually as backpropagation goes down the layers during the reverse pass). We are up to a total of roughly 500.0 + 34.3 + 3.4 = 537.7 MB. And that's really an optimistic bare minimum.

3. If your GPU runs out of memory while training a CNN, here are five things you could try to solve the problem (other than purchasing a GPU with more RAM):

   - Reduce the mini-batch size.
   - Reduce dimensionality using a larger stride in one or more layers.
   - Remove one or more layers.
   - Use 16-bit floats instead of 32-bit floats.
   - Distribute the CNN across multiple devices.

4. A max pooling layer has no parameters at all, whereas a convolutional layer has quite a few (see the previous questions).
5. A *local response normalization* layer makes the neurons that most strongly activate inhibit neurons at the same location but in neighboring feature maps, which encourages different feature maps to specialize and pushes them apart, forcing them to explore a wider range of features. It is typically used in the lower layers to have a larger pool of low-level features that the upper layers can build upon.
6. The main innovations in AlexNet compared to LeNet-5 are (1) it is much larger and deeper, and (2) it stacks convolutional layers directly on top of each other,

instead of stacking a pooling layer on top of each convolutional layer. The main innovation in GoogLeNet is the introduction of *inception modules*, which make it possible to have a much deeper net than previous CNN architectures, with fewer parameters. Finally, ResNet's main innovation is the introduction of skip connections, which make it possible to go well beyond 100 layers. Arguably, its simplicity and consistency are also rather innovative.

For the solutions to exercises 7, 8, 9, and 10, please see the Jupyter notebooks available at *https://github.com/ageron/handson-ml*.

# Chapter 14: Recurrent Neural Networks

1. Here are a few RNN applications:

   - For a sequence-to-sequence RNN: predicting the weather (or any other time series), machine translation (using an encoder–decoder architecture), video captioning, speech to text, music generation (or other sequence generation), identifying the chords of a song.
   - For a sequence-to-vector RNN: classifying music samples by music genre, analyzing the sentiment of a book review, predicting what word an aphasic patient is thinking of based on readings from brain implants, predicting the probability that a user will want to watch a movie based on her watch history (this is one of many possible implementations of *collaborative filtering*).
   - For a vector-to-sequence RNN: image captioning, creating a music playlist based on an embedding of the current artist, generating a melody based on a set of parameters, locating pedestrians in a picture (e.g., a video frame from a self-driving car's camera).

2. In general, if you translate a sentence one word at a time, the result will be terrible. For example, the French sentence "Je vous en prie" means "You are welcome," but if you translate it one word at a time, you get "I you in pray." Huh? It is much better to read the whole sentence first and then translate it. A plain sequence-to-sequence RNN would start translating a sentence immediately after reading the first word, while an encoder–decoder RNN will first read the whole sentence and then translate it. That said, one could imagine a plain sequence-to-sequence RNN that would output silence whenever it is unsure about what to say next (just like human translators do when they must translate a live broadcast).
3. To classify videos based on the visual content, one possible architecture could be to take (say) one frame per second, then run each frame through a convolutional neural network, feed the output of the CNN to a sequence-to-vector RNN, and finally run its output through a softmax layer, giving you all the class probabilities. For training you would just use cross entropy as the cost function. If you

wanted to use the audio for classification as well, you could convert every second of audio to a spectrograph, feed this spectrograph to a CNN, and feed the output of this CNN to the RNN (along with the corresponding output of the other CNN).

4. Building an RNN using `dynamic_rnn()` rather than `static_rnn()` offers several advantages:

   - It is based on a `while_loop()` operation that is able to swap the GPU's memory to the CPU's memory during backpropagation, avoiding out-of-memory errors.
   - It is arguably easier to use, as it can directly take a single tensor as input and output (covering all time steps), rather than a list of tensors (one per time step). No need to stack, unstack, or transpose.
   - It generates a smaller graph, easier to visualize in TensorBoard.

5. To handle variable length input sequences, the simplest option is to set the `sequence_length` parameter when calling the `static_rnn()` or `dynamic_rnn()` functions. Another option is to pad the smaller inputs (e.g., with zeros) to make them the same size as the largest input (this may be faster than the first option if the input sequences all have very similar lengths). To handle variable-length output sequences, if you know in advance the length of each output sequence, you can use the `sequence_length` parameter (for example, consider a sequence-to-sequence RNN that labels every frame in a video with a violence score: the output sequence will be exactly the same length as the input sequence). If you don't know in advance the length of the output sequence, you can use the padding trick: always output the same size sequence, but ignore any outputs that come after the end-of-sequence token (by ignoring them when computing the cost function).
6. To distribute training and execution of a deep RNN across multiple GPUs, a common technique is simply to place each layer on a different GPU (see Chapter 12).

For the solutions to exercises 7, 8, and 9, please see the Jupyter notebooks available at *https://github.com/ageron/handson-ml*.

# Chapter 15: Autoencoders

1. Here are some of the main tasks that autoencoders are used for:

   - Feature extraction
   - Unsupervised pretraining

- Dimensionality reduction
- Generative models
- Anomaly detection (an autoencoder is generally bad at reconstructing outliers)

2. If you want to train a classifier and you have plenty of unlabeled training data, but only a few thousand labeled instances, then you could first train a deep autoencoder on the full dataset (labeled + unlabeled), then reuse its lower half for the classifier (i.e., reuse the layers up to the codings layer, included) and train the classifier using the labeled data. If you have little labeled data, you probably want to freeze the reused layers when training the classifier.
3. The fact that an autoencoder perfectly reconstructs its inputs does not necessarily mean that it is a good autoencoder; perhaps it is simply an overcomplete autoencoder that learned to copy its inputs to the codings layer and then to the outputs. In fact, even if the codings layer contained a single neuron, it would be possible for a very deep autoencoder to learn to map each training instance to a different coding (e.g., the first instance could be mapped to 0.001, the second to 0.002, the third to 0.003, and so on), and it could learn "by heart" to reconstruct the right training instance for each coding. It would perfectly reconstruct its inputs without really learning any useful pattern in the data. In practice such a mapping is unlikely to happen, but it illustrates the fact that perfect reconstructions are not a guarantee that the autoencoder learned anything useful. However, if it produces very bad reconstructions, then it is almost guaranteed to be a bad autoencoder. To evaluate the performance of an autoencoder, one option is to measure the reconstruction loss (e.g., compute the MSE, the mean square of the outputs minus the inputs). Again, a high reconstruction loss is a good sign that the autoencoder is bad, but a low reconstruction loss is not a guarantee that it is good. You should also evaluate the autoencoder according to what it will be used for. For example, if you are using it for unsupervised pretraining of a classifier, then you should also evaluate the classifier's performance.
4. An undercomplete autoencoder is one whose codings layer is smaller than the input and output layers. If it is larger, then it is an overcomplete autoencoder. The main risk of an excessively undercomplete autoencoder is that it may fail to reconstruct the inputs. The main risk of an overcomplete autoencoder is that it may just copy the inputs to the outputs, without learning any useful feature.
5. To tie the weights of an encoder layer and its corresponding decoder layer, you simply make the decoder weights equal to the transpose of the encoder weights. This reduces the number of parameters in the model by half, often making training converge faster with less training data, and reducing the risk of overfitting the training set.
6. To visualize the features learned by the lower layer of a stacked autoencoder, a common technique is simply to plot the weights of each neuron, by reshaping

each weight vector to the size of an input image (e.g., for MNIST, reshaping a weight vector of shape `[784]` to `[28, 28]`). To visualize the features learned by higher layers, one technique is to display the training instances that most activate each neuron.

7. A generative model is a model capable of randomly generating outputs that resemble the training instances. For example, once trained successfully on the MNIST dataset, a generative model can be used to randomly generate realistic images of digits. The output distribution is typically similar to the training data. For example, since MNIST contains many images of each digit, the generative model would output roughly the same number of images of each digit. Some generative models can be parametrized—for example, to generate only some kinds of outputs. An example of a generative autoencoder is the variational autoencoder.

For the solutions to exercises 8, 9, and 10, please see the Jupyter notebooks available at *https://github.com/ageron/handson-ml*.

# Chapter 16: Reinforcement Learning

1. Reinforcement Learning is an area of Machine Learning aimed at creating agents capable of taking actions in an environment in a way that maximizes rewards over time. There are many differences between RL and regular supervised and unsupervised learning. Here are a few:

   - In supervised and unsupervised learning, the goal is generally to find patterns in the data and use them to make predictions. In Reinforcement Learning, the goal is to find a good policy.
   - Unlike in supervised learning, the agent is not explicitly given the "right" answer. It must learn by trial and error.
   - Unlike in unsupervised learning, there is a form of supervision, through rewards. We do not tell the agent how to perform the task, but we do tell it when it is making progress or when it is failing.
   - A Reinforcement Learning agent needs to find the right balance between exploring the environment, looking for new ways of getting rewards, and exploiting sources of rewards that it already knows. In contrast, supervised and unsupervised learning systems generally don't need to worry about exploration; they just feed on the training data they are given.
   - In supervised and unsupervised learning, training instances are typically independent (in fact, they are generally shuffled). In Reinforcement Learning, consecutive observations are generally *not* independent. An agent may remain in the same region of the environment for a while before it moves on, so consecu-

tive observations will be very correlated. In some cases a replay memory is used to ensure that the training algorithm gets fairly independent observations.

2. Here are a few possible applications of Reinforcement Learning, other than those mentioned in Chapter 16:

   *Music personalization*
   : The environment is a user's personalized web radio. The agent is the software deciding what song to play next for that user. Its possible actions are to play any song in the catalog (it must try to choose a song the user will enjoy) or to play an advertisement (it must try to choose an ad that the user will be interested in). It gets a small reward every time the user listens to a song, a larger reward every time the user listens to an ad, a negative reward when the user skips a song or an ad, and a very negative reward if the user leaves.

   *Marketing*
   : The environment is your company's marketing department. The agent is the software that defines which customers a mailing campaign should be sent to, given their profile and purchase history (for each customer it has two possible actions: send or don't send). It gets a negative reward for the cost of the mailing campaign, and a positive reward for estimated revenue generated from this campaign.

   *Product delivery*
   : Let the agent control a fleet of delivery trucks, deciding what they should pick up at the depots, where they should go, what they should drop off, and so on. They would get positive rewards for each product delivered on time, and negative rewards for late deliveries.

3. When estimating the value of an action, Reinforcement Learning algorithms typically sum all the rewards that this action led to, giving more weight to immediate rewards, and less weight to later rewards (considering that an action has more influence on the near future than on the distant future). To model this, a discount rate is typically applied at each time step. For example, with a discount rate of 0.9, a reward of 100 that is received two time steps later is counted as only $0.9^2 \times 100 = 81$ when you are estimating the value of the action. You can think of the discount rate as a measure of how much the future is valued relative to the present: if it is very close to 1, then the future is valued almost as much as the present. If it is close to 0, then only immediate rewards matter. Of course, this impacts the optimal policy tremendously: if you value the future, you may be willing to put up with a lot of immediate pain for the prospect of eventual rewards, while if you don't value the future, you will just grab any immediate reward you can find, never investing in the future.

4. To measure the performance of a Reinforcement Learning agent, you can simply sum up the rewards it gets. In a simulated environment, you can run many episodes and look at the total rewards it gets on average (and possibly look at the min, max, standard deviation, and so on).
5. The credit assignment problem is the fact that when a Reinforcement Learning agent receives a reward, it has no direct way of knowing which of its previous actions contributed to this reward. It typically occurs when there is a large delay between an action and the resulting rewards (e.g., during a game of Atari's *Pong*, there may be a few dozen time steps between the moment the agent hits the ball and the moment it wins the point). One way to alleviate it is to provide the agent with shorter-term rewards, when possible. This usually requires prior knowledge about the task. For example, if we want to build an agent that will learn to play chess, instead of giving it a reward only when it wins the game, we could give it a reward every time it captures one of the opponent's pieces.
6. An agent can often remain in the same region of its environment for a while, so all of its experiences will be very similar for that period of time. This can introduce some bias in the learning algorithm. It may tune its policy for this region of the environment, but it will not perform well as soon as it moves out of this region. To solve this problem, you can use a replay memory; instead of using only the most immediate experiences for learning, the agent will learn based on a buffer of its past experiences, recent and not so recent (perhaps this is why we dream at night: to replay our experiences of the day and better learn from them?).
7. An off-policy RL algorithm learns the value of the optimal policy (i.e., the sum of discounted rewards that can be expected for each state if the agent acts optimally) while the agent follows a different policy. Q-Learning is a good example of such an algorithm. In contrast, an on-policy algorithm learns the value of the policy that the agent actually executes, including both exploration and exploitation.

For the solutions to exercises 8, 9, and 10, please see the Jupyter notebooks available at *https://github.com/ageron/handson-ml*.

APPENDIX B

# Machine Learning Project Checklist

This checklist can guide you through your Machine Learning projects. There are eight main steps:

1. Frame the problem and look at the big picture.
2. Get the data.
3. Explore the data to gain insights.
4. Prepare the data to better expose the underlying data patterns to Machine Learning algorithms.
5. Explore many different models and short-list the best ones.
6. Fine-tune your models and combine them into a great solution.
7. Present your solution.
8. Launch, monitor, and maintain your system.

Obviously, you should feel free to adapt this checklist to your needs.

## Frame the Problem and Look at the Big Picture

1. Define the objective in business terms.
2. How will your solution be used?
3. What are the current solutions/workarounds (if any)?
4. How should you frame this problem (supervised/unsupervised, online/offline, etc.)?
5. How should performance be measured?
6. Is the performance measure aligned with the business objective?

7. What would be the minimum performance needed to reach the business objective?
8. What are comparable problems? Can you reuse experience or tools?
9. Is human expertise available?
10. How would you solve the problem manually?
11. List the assumptions you (or others) have made so far.
12. Verify assumptions if possible.

## Get the Data

Note: automate as much as possible so you can easily get fresh data.

1. List the data you need and how much you need.
2. Find and document where you can get that data.
3. Check how much space it will take.
4. Check legal obligations, and get authorization if necessary.
5. Get access authorizations.
6. Create a workspace (with enough storage space).
7. Get the data.
8. Convert the data to a format you can easily manipulate (without changing the data itself).
9. Ensure sensitive information is deleted or protected (e.g., anonymized).
10. Check the size and type of data (time series, sample, geographical, etc.).
11. Sample a test set, put it aside, and never look at it (no data snooping!).

## Explore the Data

Note: try to get insights from a field expert for these steps.

1. Create a copy of the data for exploration (sampling it down to a manageable size if necessary).
2. Create a Jupyter notebook to keep a record of your data exploration.
3. Study each attribute and its characteristics:
    - Name
    - Type (categorical, int/float, bounded/unbounded, text, structured, etc.)

- % of missing values
- Noisiness and type of noise (stochastic, outliers, rounding errors, etc.)
- Possibly useful for the task?
- Type of distribution (Gaussian, uniform, logarithmic, etc.)

4. For supervised learning tasks, identify the target attribute(s).
5. Visualize the data.
6. Study the correlations between attributes.
7. Study how you would solve the problem manually.
8. Identify the promising transformations you may want to apply.
9. Identify extra data that would be useful (go back to "Get the Data" on page 504).
10. Document what you have learned.

## Prepare the Data

Notes:

- Work on copies of the data (keep the original dataset intact).
- Write functions for all data transformations you apply, for five reasons:
  - So you can easily prepare the data the next time you get a fresh dataset
  - So you can apply these transformations in future projects
  - To clean and prepare the test set
  - To clean and prepare new data instances once your solution is live
  - To make it easy to treat your preparation choices as hyperparameters

1. Data cleaning:
   - Fix or remove outliers (optional).
   - Fill in missing values (e.g., with zero, mean, median…) or drop their rows (or columns).
2. Feature selection (optional):
   - Drop the attributes that provide no useful information for the task.
3. Feature engineering, where appropriate:
   - Discretize continuous features.

- Decompose features (e.g., categorical, date/time, etc.).
- Add promising transformations of features (e.g., log($x$), sqrt($x$), $x^2$, etc.).
- Aggregate features into promising new features.

4. Feature scaling: standardize or normalize features.

## Short-List Promising Models

Notes:

- If the data is huge, you may want to sample smaller training sets so you can train many different models in a reasonable time (be aware that this penalizes complex models such as large neural nets or Random Forests).
- Once again, try to automate these steps as much as possible.

1. Train many quick and dirty models from different categories (e.g., linear, naive Bayes, SVM, Random Forests, neural net, etc.) using standard parameters.
2. Measure and compare their performance.
   - For each model, use $N$-fold cross-validation and compute the mean and standard deviation of the performance measure on the $N$ folds.
3. Analyze the most significant variables for each algorithm.
4. Analyze the types of errors the models make.
   - What data would a human have used to avoid these errors?
5. Have a quick round of feature selection and engineering.
6. Have one or two more quick iterations of the five previous steps.
7. Short-list the top three to five most promising models, preferring models that make different types of errors.

## Fine-Tune the System

Notes:

- You will want to use as much data as possible for this step, especially as you move toward the end of fine-tuning.
- As always automate what you can.

1. Fine-tune the hyperparameters using cross-validation.
   - Treat your data transformation choices as hyperparameters, especially when you are not sure about them (e.g., should I replace missing values with zero or with the median value? Or just drop the rows?).
   - Unless there are very few hyperparameter values to explore, prefer random search over grid search. If training is very long, you may prefer a Bayesian optimization approach (e.g., using Gaussian process priors, as described by Jasper Snoek, Hugo Larochelle, and Ryan Adams).[1]
2. Try Ensemble methods. Combining your best models will often perform better than running them individually.
3. Once you are confident about your final model, measure its performance on the test set to estimate the generalization error.



Don't tweak your model after measuring the generalization error: you would just start overfitting the test set.

## Present Your Solution

1. Document what you have done.
2. Create a nice presentation.
   - Make sure you highlight the big picture first.
3. Explain why your solution achieves the business objective.
4. Don't forget to present interesting points you noticed along the way.
   - Describe what worked and what did not.
   - List your assumptions and your system's limitations.
5. Ensure your key findings are communicated through beautiful visualizations or easy-to-remember statements (e.g., "the median income is the number-one predictor of housing prices").

1 "Practical Bayesian Optimization of Machine Learning Algorithms," J. Snoek, H. Larochelle, R. Adams (2012).

## Launch!

1. Get your solution ready for production (plug into production data inputs, write unit tests, etc.).
2. Write monitoring code to check your system's live performance at regular intervals and trigger alerts when it drops.
   - Beware of slow degradation too: models tend to "rot" as data evolves.
   - Measuring performance may require a human pipeline (e.g., via a crowdsourcing service).
   - Also monitor your inputs' quality (e.g., a malfunctioning sensor sending random values, or another team's output becoming stale). This is particularly important for online learning systems.
3. Retrain your models on a regular basis on fresh data (automate as much as possible).

# APPENDIX C
# SVM Dual Problem

To understand *duality*, you first need to understand the *Lagrange multipliers* method. The general idea is to transform a constrained optimization objective into an unconstrained one, by moving the constraints into the objective function. Let's look at a simple example. Suppose you want to find the values of $x$ and $y$ that minimize the function $f(x,y) = x^2 + 2y$, subject to an *equality constraint*: $3x + 2y + 1 = 0$. Using the Lagrange multipliers method, we start by defining a new function called the *Lagrangian* (or *Lagrange function*): $g(x, y, \alpha) = f(x, y) - \alpha(3x + 2y + 1)$. Each constraint (in this case just one) is subtracted from the original objective, multiplied by a new variable called a Lagrange multiplier.

Joseph-Louis Lagrange showed that if $(\hat{x}, \hat{y})$ is a solution to the constrained optimization problem, then there must exist an $\hat{\alpha}$ such that $(\hat{x}, \hat{y}, \hat{\alpha})$ is a *stationary point* of the Lagrangian (a stationary point is a point where all partial derivatives are equal to zero). In other words, we can compute the partial derivatives of $g(x, y, \alpha)$ with regards to $x$, $y$, and $\alpha$; we can find the points where these derivatives are all equal to zero; and the solutions to the constrained optimization problem (if they exist) must be among these stationary points.

In this example the partial derivatives are: $\begin{cases} \frac{\partial}{\partial x} g(x, y, \alpha) = 2x - 3\alpha \\ \frac{\partial}{\partial y} g(x, y, \alpha) = 2 - 2\alpha \\ \frac{\partial}{\partial \alpha} g(x, y, \alpha) = -3x - 2y - 1 \end{cases}$

When all these partial derivatives are equal to 0, we find that $2\hat{x} - 3\hat{\alpha} = 2 - 2\hat{\alpha} = -3\hat{x} - 2\hat{y} - 1 = 0$, from which we can easily find that $\hat{x} = \frac{3}{2}$, $\hat{y} = -\frac{11}{4}$, and $\hat{\alpha} = 1$. This is the only stationary point, and as it respects the constraint, it must be the solution to the constrained optimization problem.

However, this method applies only to equality constraints. Fortunately, under some regularity conditions (which are respected by the SVM objectives), this method can be generalized to *inequality constraints* as well (e.g., $3x + 2y + 1 \geq 0$). The *generalized Lagrangian* for the hard margin problem is given by Equation C-1, where the $\alpha^{(i)}$ variables are called the *Karush–Kuhn–Tucker* (KKT) multipliers, and they must be greater or equal to zero.

*Equation C-1. Generalized Lagrangian for the hard margin problem*

$$\mathcal{L}(\mathbf{w}, b, \alpha) = \frac{1}{2}\mathbf{w}^T \cdot \mathbf{w} - \sum_{i=1}^{m} \alpha^{(i)}\left(t^{(i)}\left(\mathbf{w}^T \cdot \mathbf{x}^{(i)} + b\right) - 1\right)$$

$$\text{with} \quad \alpha^{(i)} \geq 0 \quad \text{for } i = 1, 2, \cdots, m$$

Just like with the Lagrange multipliers method, you can compute the partial derivatives and locate the stationary points. If there is a solution, it will necessarily be among the stationary points $\left(\widehat{\mathbf{w}}, \hat{b}, \hat{\alpha}\right)$ that respect the *KKT conditions*:

- Respect the problem's constraints: $t^{(i)}\left(\left(\widehat{\mathbf{w}}\right)^T \cdot \mathbf{x}^{(i)} + \hat{b}\right) \geq 1 \quad \text{for } i = 1, 2, \cdots, m$,
- Verify $\hat{\alpha}^{(i)} \geq 0 \quad \text{for } i = 1, 2, \cdots, m$,
- Either $\hat{\alpha}^{(i)} = 0$ or the $i^{th}$ constraint must be an *active constraint*, meaning it must hold by equality: $t^{(i)}\left(\left(\widehat{\mathbf{w}}\right)^T \cdot \mathbf{x}^{(i)} + \hat{b}\right) = 1$. This condition is called the *complementary slackness* condition. It implies that either $\hat{\alpha}^{(i)} = 0$ or the $i^{th}$ instance lies on the boundary (it is a support vector).

Note that the KKT conditions are necessary conditions for a stationary point to be a solution of the constrained optimization problem. Under some conditions, they are also sufficient conditions. Luckily, the SVM optimization problem happens to meet these conditions, so any stationary point that meets the KKT conditions is guaranteed to be a solution to the constrained optimization problem.

We can compute the partial derivatives of the generalized Lagrangian with regards to $\mathbf{w}$ and $b$ with Equation C-2.

*Equation C-2. Partial derivatives of the generalized Lagrangian*

$$\nabla_{\mathbf{w}}\mathcal{L}(\mathbf{w}, b, \alpha) = \mathbf{w} - \sum_{i=1}^{m} \alpha^{(i)} t^{(i)} \mathbf{x}^{(i)}$$

$$\frac{\partial}{\partial b}\mathcal{L}(\mathbf{w}, b, \alpha) = -\sum_{i=1}^{m} \alpha^{(i)} t^{(i)}$$

When these partial derivatives are equal to 0, we have Equation C-3.

*Equation C-3. Properties of the stationary points*

$$\widehat{\mathbf{w}} = \sum_{i=1}^{m} \widehat{\alpha}^{(i)} t^{(i)} \mathbf{x}^{(i)}$$

$$\sum_{i=1}^{m} \widehat{\alpha}^{(i)} t^{(i)} = 0$$

If we plug these results into the definition of the generalized Lagrangian, some terms disappear and we find Equation C-4.

*Equation C-4. Dual form of the SVM problem*

$$\mathcal{L}\left(\widehat{\mathbf{w}}, \hat{b}, \alpha\right) = \frac{1}{2} \sum_{i=1}^{m} \sum_{j=1}^{m} \alpha^{(i)} \alpha^{(j)} t^{(i)} t^{(j)} \mathbf{x}^{(i)T} \cdot \mathbf{x}^{(j)} \quad - \quad \sum_{i=1}^{m} \alpha^{(i)}$$

$$\text{with} \quad \alpha^{(i)} \geq 0 \quad \text{for } i = 1, 2, \cdots, m$$

The goal is now to find the vector $\widehat{\boldsymbol{\alpha}}$ that minimizes this function, with $\widehat{\alpha}^{(i)} \geq 0$ for all instances. This constrained optimization problem is the dual problem we were looking for.

Once you find the optimal $\widehat{\boldsymbol{\alpha}}$, you can compute $\widehat{\mathbf{w}}$ using the first line of Equation C-3. To compute $\hat{b}$, you can use the fact that a support vector must verify $t^{(i)}(\widehat{\mathbf{w}}^T \cdot \mathbf{x}^{(i)} + \hat{b}) = 1$, so if the k$^{\text{th}}$ instance is a support vector (i.e., $\widehat{\alpha}^{(k)} > 0$), you can use it to compute $\hat{b} = t^{(k)} - \widehat{\mathbf{w}}^T \cdot \mathbf{x}^{(k)}$. However, it is often prefered to compute the average over all support vectors to get a more stable and precise value, as in Equation C-5.

*Equation C-5. Bias term estimation using the dual form*

$$\hat{b} = \frac{1}{n_s} \sum_{\substack{i=1 \\ \widehat{\alpha}^{(i)} > 0}}^{m} \left[ t^{(i)} - \widehat{\mathbf{w}}^T \cdot \mathbf{x}^{(i)} \right]$$

# APPENDIX D
# Autodiff

This appendix explains how TensorFlow's autodiff feature works, and how it compares to other solutions.

Suppose you define a function $f(x,y) = x^2y + y + 2$, and you need its partial derivatives $\frac{\partial f}{\partial x}$ and $\frac{\partial f}{\partial y}$, typically to perform Gradient Descent (or some other optimization algorithm). Your main options are manual differentiation, symbolic differentiation, numerical differentiation, forward-mode autodiff, and finally reverse-mode autodiff. TensorFlow implements this last option. Let's go through each of these options.

## Manual Differentiation

The first approach is to pick up a pencil and a piece of paper and use your calculus knowledge to derive the partial derivatives manually. For the function $f(x,y)$ just defined, it is not too hard; you just need to use five rules:

- The derivative of a constant is 0.
- The derivative of $\lambda x$ is $\lambda$ (where $\lambda$ is a constant).
- The derivative of $x^\lambda$ is $\lambda x^{\lambda - 1}$, so the derivative of $x^2$ is $2x$.
- The derivative of a sum of functions is the sum of these functions' derivatives.
- The derivative of $\lambda$ times a function is $\lambda$ times its derivative.

From these rules, you can derive Equation D-1:

*Equation D-1. Partial derivatives of f(x,y)*

$$\frac{\partial f}{\partial x} = \frac{\partial\left(x^2 y\right)}{\partial x} + \frac{\partial y}{\partial x} + \frac{\partial 2}{\partial x} = y\frac{\partial\left(x^2\right)}{\partial x} + 0 + 0 = 2xy$$

$$\frac{\partial f}{\partial y} = \frac{\partial\left(x^2 y\right)}{\partial y} + \frac{\partial y}{\partial y} + \frac{\partial 2}{\partial y} = x^2 + 1 + 0 = x^2 + 1$$

This approach can become very tedious for more complex functions, and you run the risk of making mistakes. The good news is that deriving the mathematical equations for the partial derivatives like we just did can be automated, through a process called *symbolic differentiation*.

# Symbolic Differentiation

Figure D-1 shows how symbolic differentiation works on an even simpler function, $g(x,y) = 5 + xy$. The graph for that function is represented on the left. After symbolic differentiation, we get the graph on the right, which represents the partial derivative $\frac{\partial g}{\partial x} = 0 + (0 \times x + y \times 1) = y$ (we could similarly obtain the partial derivative with regards to $y$).

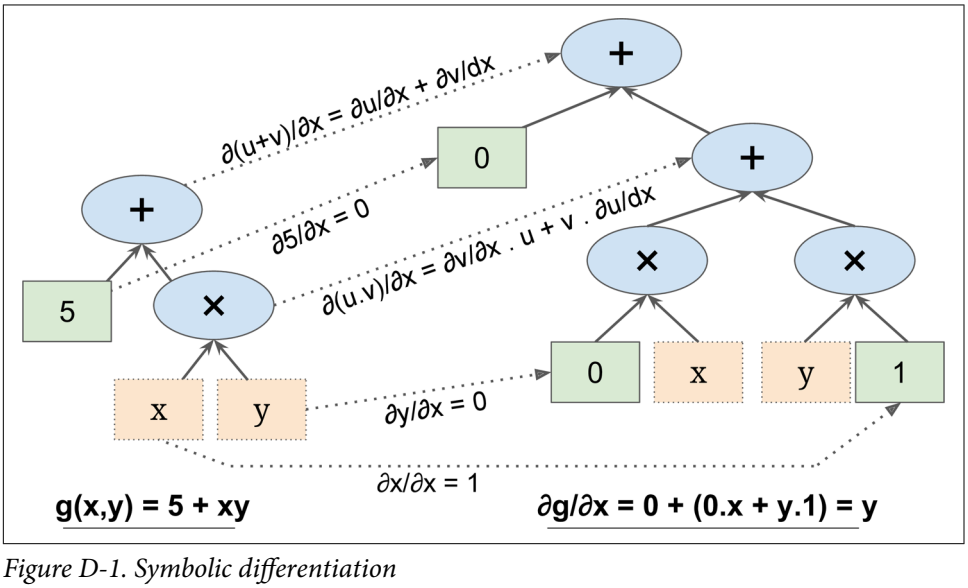



*Figure D-1. Symbolic differentiation*

The algorithm starts by getting the partial derivative of the leaf nodes. The constant node (5) returns the constant 0, since the derivative of a constant is always 0. The

variable $x$ returns the constant 1 since $\frac{\partial x}{\partial x} = 1$, and the variable $y$ returns the constant 0 since $\frac{\partial y}{\partial x} = 0$ (if we were looking for the partial derivative with regards to $y$, it would be the reverse).

Now we have all we need to move up the graph to the multiplication node in function $g$. Calculus tells us that the derivative of the product of two functions $u$ and $v$ is $\frac{\partial(u \times v)}{\partial x} = \frac{\partial v}{\partial x} \times u + v \times \frac{\partial u}{\partial x}$. We can therefore construct a large part of the graph on the right, representing $0 \times x + y \times 1$.

Finally, we can go up to the addition node in function $g$. As mentioned, the derivative of a sum of functions is the sum of these functions' derivatives. So we just need to create an addition node and connect it to the parts of the graph we have already computed. We get the correct partial derivative: $\frac{\partial g}{\partial x} = 0 + (0 \times x + y \times 1)$.

However, it can be simplified (a lot). A few trivial pruning steps can be applied to this graph to get rid of all unnecessary operations, and we get a much smaller graph with just one node: $\frac{\partial g}{\partial x} = y$.

In this case, simplification is fairly easy, but for a more complex function, symbolic differentiation can produce a huge graph that may be tough to simplify and lead to suboptimal performance. Most importantly, symbolic differentiation cannot deal with functions defined with arbitrary code—for example, the following function discussed in Chapter 9:

```
def my_func(a, b):
    z = 0
    for i in range(100):
        z = a * np.cos(z + i) + z * np.sin(b - i)
    return z
```

# Numerical Differentiation

The simplest solution is to compute an approximation of the derivatives, numerically. Recall that the derivative $h'(x_0)$ of a function $h(x)$ at a point $x_0$ is the slope of the function at that point, or more precisely Equation D-2.

*Equation D-2. Derivative of a function h(x) at point $x_0$*

$$h'(x_0) = \lim_{x \to x_0} \frac{h(x) - h(x_0)}{x - x_0}$$

$$= \lim_{\epsilon \to 0} \frac{h(x_0 + \epsilon) - h(x_0)}{\epsilon}$$

So if we want to calculate the partial derivative of $f(x,y)$ with regards to $x$, at $x = 3$ and $y = 4$, we can simply compute $f(3 + \epsilon, 4) - f(3, 4)$ and divide the result by $\epsilon$, using a very small value for $\epsilon$. That's exactly what the following code does:

```
def f(x, y):
    return x**2*y + y + 2

def derivative(f, x, y, x_eps, y_eps):
    return (f(x + x_eps, y + y_eps) - f(x, y)) / (x_eps + y_eps)

df_dx = derivative(f, 3, 4, 0.00001, 0)
df_dy = derivative(f, 3, 4, 0, 0.00001)
```

Unfortunately, the result is imprecise (and it gets worse for more complex functions). The correct results are respectively 24 and 10, but instead we get:

```
>>> print(df_dx)
24.000039999805264
>>> print(df_dy)
10.000000000331966
```

Notice that to compute both partial derivatives, we have to call `f()` at least three times (we called it four times in the preceding code, but it could be optimized). If there were 1,000 parameters, we would need to call `f()` at least 1,001 times. When you are dealing with large neural networks, this makes numerical differentiation way too inefficient.

However, numerical differentiation is so simple to implement that it is a great tool to check that the other methods are implemented correctly. For example, if it disagrees with your manually derived function, then your function probably contains a mistake.

# Forward-Mode Autodiff

*Forward-mode autodiff* is neither numerical differentiation nor symbolic differentiation, but in some ways it is their love child. It relies on *dual numbers*, which are (weird but fascinating) numbers of the form $a + b\epsilon$ where $a$ and $b$ are real numbers and $\epsilon$ is an infinitesimal number such that $\epsilon^2 = 0$ (but $\epsilon \neq 0$). You can think of the dual number $42 + 24\epsilon$ as something akin to $42.0000\cdots000024$ with an infinite number of 0s (but of course this is simplified just to give you some idea of what dual numbers are). A dual number is represented in memory as a pair of floats. For example, $42 + 24\epsilon$ is represented by the pair (42.0, 24.0).

Dual numbers can be added, multiplied, and so on, as shown in Equation D-3.

*Equation D-3. A few operations with dual numbers*

$$\lambda(a + b\epsilon) = \lambda a + \lambda b\epsilon$$
$$(a + b\epsilon) + (c + d\epsilon) = (a + c) + (b + d)\epsilon$$
$$(a + b\epsilon) \times (c + d\epsilon) = ac + (ad + bc)\epsilon + (bd)\epsilon^2 = ac + (ad + bc)\epsilon$$

Most importantly, it can be shown that $h(a + b\epsilon) = h(a) + b \times h'(a)\epsilon$, so computing $h(a + \epsilon)$ gives you both $h(a)$ and the derivative $h'(a)$ in just one shot. Figure D-2 shows how forward-mode autodiff computes the partial derivative of $f(x,y)$ with regards to $x$ at $x = 3$ and $y = 4$. All we need to do is compute $f(3 + \epsilon, 4)$; this will output a dual number whose first component is equal to $f(3, 4)$ and whose second component is equal to $\frac{\partial f}{\partial x}(3, 4)$.

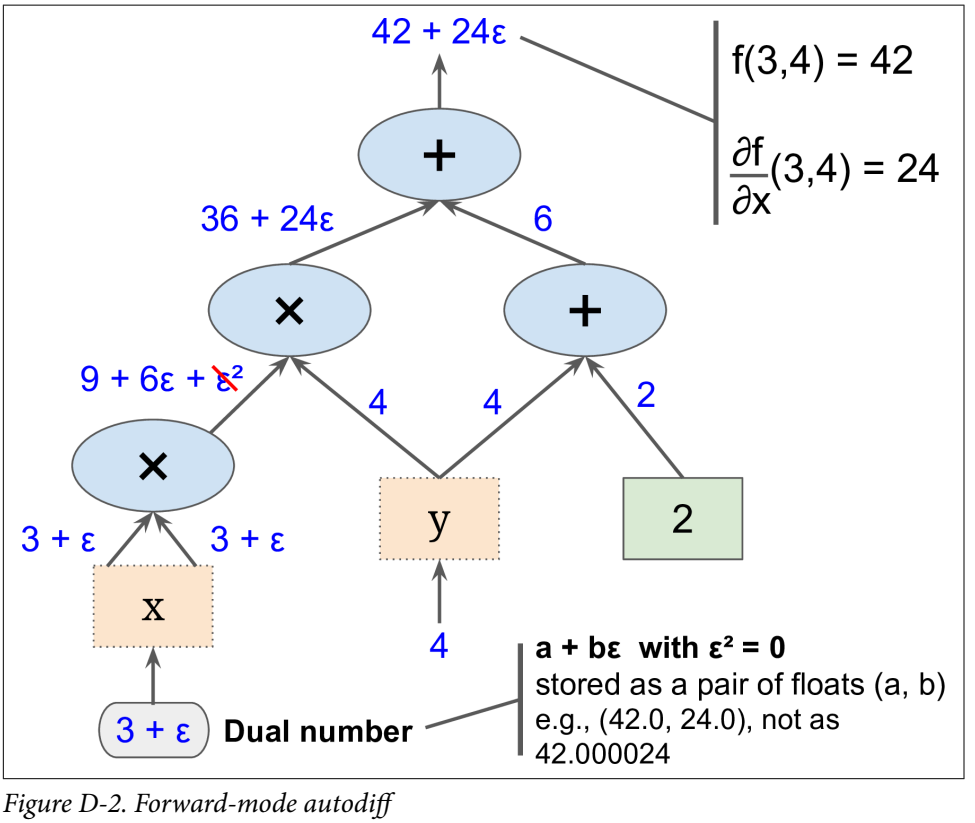



*Figure D-2. Forward-mode autodiff*

To compute $\frac{\partial f}{\partial y}(3, 4)$ we would have to go through the graph again, but this time with $x = 3$ and $y = 4 + \epsilon$.

So forward-mode autodiff is much more accurate than numerical differentiation, but it suffers from the same major flaw: if there were 1,000 parameters, it would require 1,000 passes through the graph to compute all the partial derivatives. This is where reverse-mode autodiff shines: it can compute all of them in just two passes through the graph.

# Reverse-Mode Autodiff

Reverse-mode autodiff is the solution implemented by TensorFlow. It first goes through the graph in the forward direction (i.e., from the inputs to the output) to compute the value of each node. Then it does a second pass, this time in the reverse direction (i.e., from the output to the inputs) to compute all the partial derivatives. Figure D-3 represents the second pass. During the first pass, all the node values were computed, starting from $x = 3$ and $y = 4$. You can see those values at the bottom right of each node (e.g., $x \times x = 9$). The nodes are labeled $n_1$ to $n_7$ for clarity. The output node is $n_7$: $f(3,4) = n_7 = 42$.

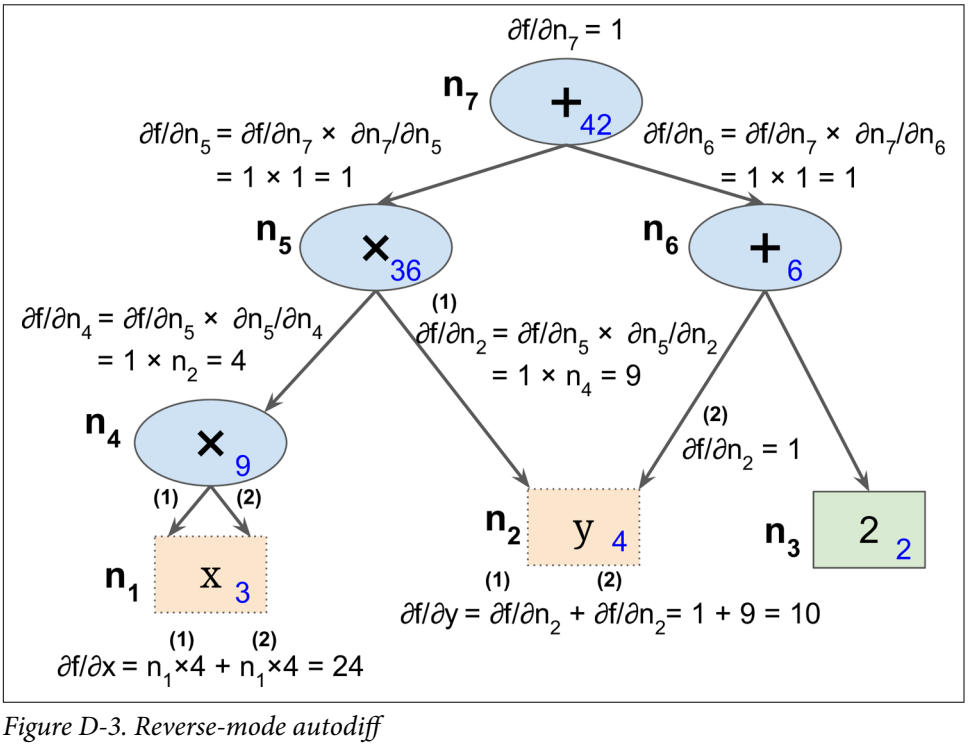



*Figure D-3. Reverse-mode autodiff*

The idea is to gradually go down the graph, computing the partial derivative of $f(x,y)$ with regards to each consecutive node, until we reach the variable nodes. For this, reverse-mode autodiff relies heavily on the *chain rule*, shown in Equation D-4.

*Equation D-4. Chain rule*

$$\frac{\partial f}{\partial x} = \frac{\partial f}{\partial n_i} \times \frac{\partial n_i}{\partial x}$$

Since $n_7$ is the output node, $f = n_7$ so trivially $\frac{\partial f}{\partial n_7} = 1$.

Let's continue down the graph to $n_5$: how much does $f$ vary when $n_5$ varies? The answer is $\frac{\partial f}{\partial n_5} = \frac{\partial f}{\partial n_7} \times \frac{\partial n_7}{\partial n_5}$. We already know that $\frac{\partial f}{\partial n_7} = 1$, so all we need is $\frac{\partial n_7}{\partial n_5}$. Since $n_7$ simply performs the sum $n_5 + n_6$, we find that $\frac{\partial n_7}{\partial n_5} = 1$, so $\frac{\partial f}{\partial n_5} = 1 \times 1 = 1$.

Now we can proceed to node $n_4$: how much does $f$ vary when $n_4$ varies? The answer is $\frac{\partial f}{\partial n_4} = \frac{\partial f}{\partial n_5} \times \frac{\partial n_5}{\partial n_4}$. Since $n_5 = n_4 \times n_2$, we find that $\frac{\partial n_5}{\partial n_4} = n_2$, so $\frac{\partial f}{\partial n_4} = 1 \times n_2 = 4$.

The process continues until we reach the bottom of the graph. At that point we will have calculated all the partial derivatives of $f(x,y)$ at the point $x = 3$ and $y = 4$. In this example, we find $\frac{\partial f}{\partial x} = 24$ and $\frac{\partial f}{\partial y} = 10$. Sounds about right!

Reverse-mode autodiff is a very powerful and accurate technique, especially when there are many inputs and few outputs, since it requires only one forward pass plus one reverse pass per output to compute all the partial derivatives for all outputs with regards to all the inputs. Most importantly, it can deal with functions defined by arbitrary code. It can also handle functions that are not entirely differentiable, as long as you ask it to compute the partial derivatives at points that are differentiable.



If you implement a new type of operation in TensorFlow and you want to make it compatible with autodiff, then you need to provide a function that builds a subgraph to compute its partial derivatives with regards to its inputs. For example, suppose you implement a function that computes the square of its input $f(x) = x^2$. In that case you would need to provide the corresponding derivative function $f'(x) = 2x$. Note that this function does not compute a numerical result, but instead builds a subgraph that will (later) compute the result. This is very useful because it means that you can compute gradients of gradients (to compute second-order derivatives, or even higher-order derivatives).

APPENDIX E

# Other Popular ANN Architectures

In this appendix we will give a quick overview of a few historically important neural network architectures that are much less used today than deep Multi-Layer Perceptrons (Chapter 10), convolutional neural networks (Chapter 13), recurrent neural networks (Chapter 14), or autoencoders (Chapter 15). They are often mentioned in the literature, and some are still used in many applications, so it is worth knowing about them. Moreover, we will discuss *deep belief nets* (DBNs), which were the state of the art in Deep Learning until the early 2010s. They are still the subject of very active research, so they may well come back with a vengeance in the near future.

## Hopfield Networks

*Hopfield networks* were first introduced by W. A. Little in 1974, then popularized by J. Hopfield in 1982. They are *associative memory* networks: you first teach them some patterns, and then when they see a new pattern they (hopefully) output the closest learned pattern. This has made them useful in particular for character recognition before they were outperformed by other approaches. You first train the network by showing it examples of character images (each binary pixel maps to one neuron), and then when you show it a new character image, after a few iterations it outputs the closest learned character.

They are fully connected graphs (see Figure E-1); that is, every neuron is connected to every other neuron. Note that on the diagram the images are $6 \times 6$ pixels, so the neural network on the left should contain 36 neurons (and 648 connections), but for visual clarity a much smaller network is represented.

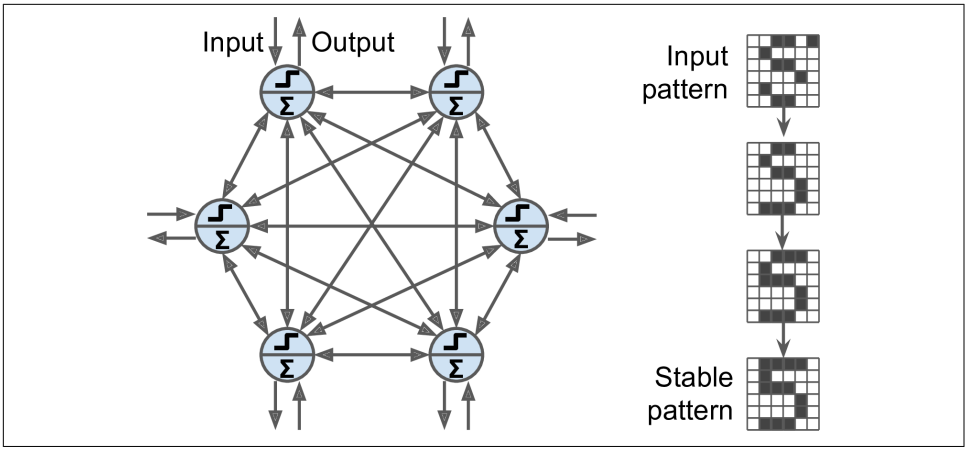



*Figure E-1. Hopfield network*

The training algorithm works by using Hebb's rule: for each training image, the weight between two neurons is increased if the corresponding pixels are both on or both off, but decreased if one pixel is on and the other is off.

To show a new image to the network, you just activate the neurons that correspond to active pixels. The network then computes the output of every neuron, and this gives you a new image. You can then take this new image and repeat the whole process. After a while, the network reaches a stable state. Generally, this corresponds to the training image that most resembles the input image.

A so-called *energy function* is associated with Hopfield nets. At each iteration, the energy decreases, so the network is guaranteed to eventually stabilize to a low-energy state. The training algorithm tweaks the weights in a way that decreases the energy level of the training patterns, so the network is likely to stabilize in one of these low-energy configurations. Unfortunately, some patterns that were not in the training set also end up with low energy, so the network sometimes stabilizes in a configuration that was not learned. These are called *spurious patterns*.

Another major flaw with Hopfield nets is that they don't scale very well—their memory capacity is roughly equal to 14% of the number of neurons. For example, to classify 28 × 28 images, you would need a Hopfield net with 784 fully connected neurons and 306,936 weights. Such a network would only be able to learn about 110 different characters (14% of 784). That's a lot of parameters for such a small memory.

# Boltzmann Machines

*Boltzmann machines* were invented in 1985 by Geoffrey Hinton and Terrence Sejnowski. Just like Hopfield nets, they are fully connected ANNs, but they are based on *sto-*

*chastic neurons*: instead of using a deterministic step function to decide what value to output, these neurons output 1 with some probability, and 0 otherwise. The probability function that these ANNs use is based on the Boltzmann distribution (used in statistical mechanics) hence their name. Equation E-1 gives the probability that a particular neuron will output a 1.

*Equation E-1. Probability that the $i^{th}$ neuron will output 1*

$$p\left(s_i^{(\text{next step})} = 1\right) = \sigma\left(\frac{\sum_{j=1}^{N} w_{i,j} s_j + b_i}{T}\right)$$

- $s_j$ is the $j^{th}$ neuron's state (0 or 1).
- $w_{i,j}$ is the connection weight between the $i^{th}$ and $j^{th}$ neurons. Note that $w_{i,i} = 0$.
- $b_i$ is the $i^{th}$ neuron's bias term. We can implement this term by adding a bias neuron to the network.
- $N$ is the number of neurons in the network.
- $T$ is a number called the network's *temperature*; the higher the temperature, the more random the output is (i.e., the more the probability approaches 50%).
- $\sigma$ is the logistic function.

Neurons in Boltzmann machines are separated into two groups: *visible units* and *hidden units* (see Figure E-2). All neurons work in the same stochastic way, but the visible units are the ones that receive the inputs and from which outputs are read.

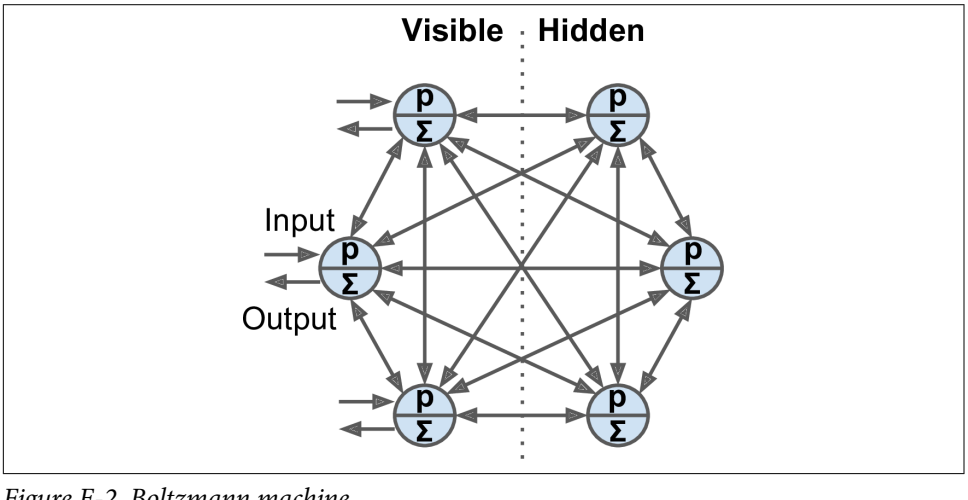



*Figure E-2. Boltzmann machine*

Because of its stochastic nature, a Boltzmann machine will never stabilize into a fixed configuration, but instead it will keep switching between many configurations. If it is left running for a sufficiently long time, the probability of observing a particular configuration will only be a function of the connection weights and bias terms, not of the original configuration (similarly, after you shuffle a deck of cards for long enough, the configuration of the deck does not depend on the initial state). When the network reaches this state where the original configuration is "forgotten," it is said to be in *thermal equilibrium* (although its configuration keeps changing all the time). By setting the network parameters appropriately, letting the network reach thermal equilibrium, and then observing its state, we can simulate a wide range of probability distributions. This is called a *generative model*.

Training a Boltzmann machine means finding the parameters that will make the network approximate the training set's probability distribution. For example, if there are three visible neurons and the training set contains 75% (0, 1, 1) triplets, 10% (0, 0, 1) triplets, and 15% (1, 1, 1) triplets, then after training a Boltzmann machine, you could use it to generate random binary triplets with about the same probability distribution. For example, about 75% of the time it would output the (0, 1, 1) triplet.

Such a generative model can be used in a variety of ways. For example, if it is trained on images, and you provide an incomplete or noisy image to the network, it will automatically "repair" the image in a reasonable way. You can also use a generative model for classification. Just add a few visible neurons to encode the training image's class (e.g., add 10 visible neurons and turn on only the fifth neuron when the training image represents a 5). Then, when given a new image, the network will automatically turn on the appropriate visible neurons, indicating the image's class (e.g., it will turn on the fifth visible neuron if the image represents a 5).

Unfortunately, there is no efficient technique to train Boltzmann machines. However, fairly efficient algorithms have been developed to train *restricted Boltzmann machines* (RBM).

## Restricted Boltzmann Machines

An RBM is simply a Boltzmann machine in which there are no connections between visible units or between hidden units, only between visible and hidden units. For example, Figure E-3 represents an RBM with three visible units and four hidden units.

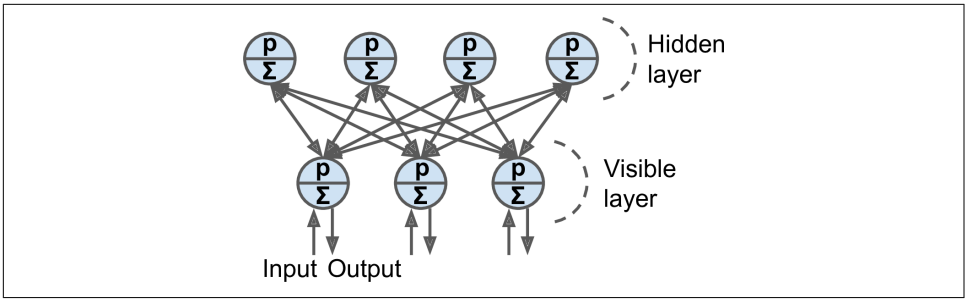



*Figure E-3. Restricted Boltzmann machine*

A very efficient training algorithm, called *Contrastive Divergence*, was introduced in 2005 by Miguel Á. Carreira-Perpiñán and Geoffrey Hinton.[1] Here is how it works: for each training instance **x**, the algorithm starts by feeding it to the network by setting the state of the visible units to $x_1, x_2, \cdots, x_n$. Then you compute the state of the hidden units by applying the stochastic equation described before (Equation E-1). This gives you a hidden vector **h** (where $h_i$ is equal to the state of the $i^{th}$ unit). Next you compute the state of the visible units, by applying the same stochastic equation. This gives you a vector **x′**. Then once again you compute the state of the hidden units, which gives you a vector **h′**. Now you can update each connection weight by applying the rule in Equation E-2, where η is the learning rate.

*Equation E-2. Contrastive divergence weight update*

$$w_{i,j} \leftarrow w_{i,j} + \eta\left(\mathbf{x} \cdot \mathbf{h}^T - \mathbf{x}' \cdot \mathbf{h}'^T\right)$$

The great benefit of this algorithm it that it does not require waiting for the network to reach thermal equilibrium: it just goes forward, backward, and forward again, and that's it. This makes it incomparably more efficient than previous algorithms, and it was a key ingredient to the first success of Deep Learning based on multiple stacked RBMs.

# Deep Belief Nets

Several layers of RBMs can be stacked; the hidden units of the first-level RBM serves as the visible units for the second-layer RBM, and so on. Such an RBM stack is called a *deep belief net* (DBN).

Yee-Whye Teh, one of Geoffrey Hinton's students, observed that it was possible to train DBNs one layer at a time using Contrastive Divergence, starting with the lower

1 "On Contrastive Divergence Learning," M. Á. Carreira-Perpiñán and G. Hinton (2005).

layers and then gradually moving up to the top layers. This led to the groundbreaking article that kickstarted the Deep Learning tsunami in 2006.[2]

Just like RBMs, DBNs learn to reproduce the probability distribution of their inputs, without any supervision. However, they are much better at it, for the same reason that deep neural networks are more powerful than shallow ones: real-world data is often organized in hierarchical patterns, and DBNs take advantage of that. Their lower layers learn low-level features in the input data, while higher layers learn high-level features.

Just like RBMs, DBNs are fundamentally unsupervised, but you can also train them in a supervised manner by adding some visible units to represent the labels. Moreover, one great feature of DBNs is that they can be trained in a semisupervised fashion. Figure E-4 represents such a DBN configured for semisupervised learning.

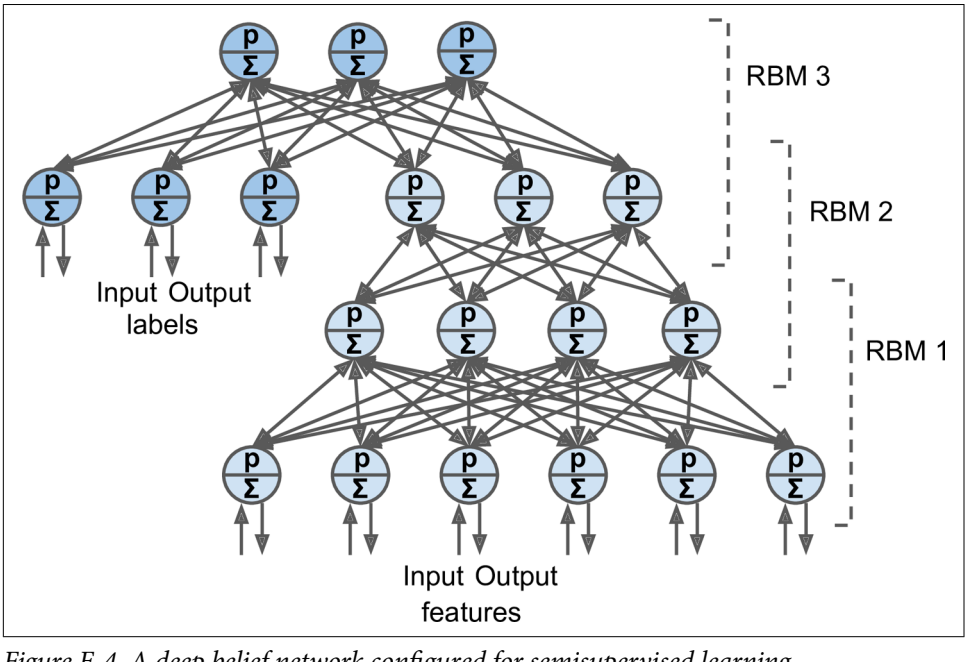



*Figure E-4. A deep belief network configured for semisupervised learning*

First, the RBM 1 is trained without supervision. It learns low-level features in the training data. Then RBM 2 is trained with RBM 1's hidden units as inputs, again without supervision: it learns higher-level features (note that RBM 2's hidden units include only the three rightmost units, not the label units). Several more RBMs could be stacked this way, but you get the idea. So far, training was 100% unsupervised.

2 "A Fast Learning Algorithm for Deep Belief Nets," G. Hinton, S. Osindero, Y. Teh (2006).

Lastly, RBM 3 is trained using both RBM 2's hidden units as inputs, as well as extra visible units used to represent the target labels (e.g., a one-hot vector representing the instance class). It learns to associate high-level features with training labels. This is the supervised step.

At the end of training, if you feed RBM 1 a new instance, the signal will propagate up to RBM 2, then up to the top of RBM 3, and then back down to the label units; hopefully, the appropriate label will light up. This is how a DBN can be used for classification.

One great benefit of this semisupervised approach is that you don't need much labeled training data. If the unsupervised RBMs do a good enough job, then only a small amount of labeled training instances per class will be necessary. Similarly, a baby learns to recognize objects without supervision, so when you point to a chair and say "chair," the baby can associate the word "chair" with the class of objects it has already learned to recognize on its own. You don't need to point to every single chair and say "chair"; only a few examples will suffice (just enough so the baby can be sure that you are indeed referring to the chair, not to its color or one of the chair's parts).

Quite amazingly, DBNs can also work in reverse. If you activate one of the label units, the signal will propagate up to the hidden units of RBM 3, then down to RBM 2, and then RBM 1, and a new instance will be output by the visible units of RBM 1. This new instance will usually look like a regular instance of the class whose label unit you activated. This generative capability of DBNs is quite powerful. For example, it has been used to automatically generate captions for images, and vice versa: first a DBN is trained (without supervision) to learn features in images, and another DBN is trained (again without supervision) to learn features in sets of captions (e.g., "car" often comes with "automobile"). Then an RBM is stacked on top of both DBNs and trained with a set of images along with their captions; it learns to associate high-level features in images with high-level features in captions. Next, if you feed the image DBN an image of a car, the signal will propagate through the network, up to the top-level RBM, and back down to the bottom of the caption DBN, producing a caption. Due to the stochastic nature of RBMs and DBNs, the caption will keep changing randomly, but it will generally be appropriate for the image. If you generate a few hundred captions, the most frequently generated ones will likely be a good description of the image.[3]

# Self-Organizing Maps

*Self-organizing maps* (SOM) are quite different from all the other types of neural networks we have discussed so far. They are used to produce a low-dimensional repre-

3 See this video by Geoffrey Hinton for more details and a demo: *http://goo.gl/7Z5QiS*.

sentation of a high-dimensional dataset, generally for visualization, clustering, or classification. The neurons are spread across a map (typically 2D for visualization, but it can be any number of dimensions you want), as shown in Figure E-5, and each neuron has a weighted connection to every input (note that the diagram shows just two inputs, but there are typically a very large number, since the whole point of SOMs is to reduce dimensionality).

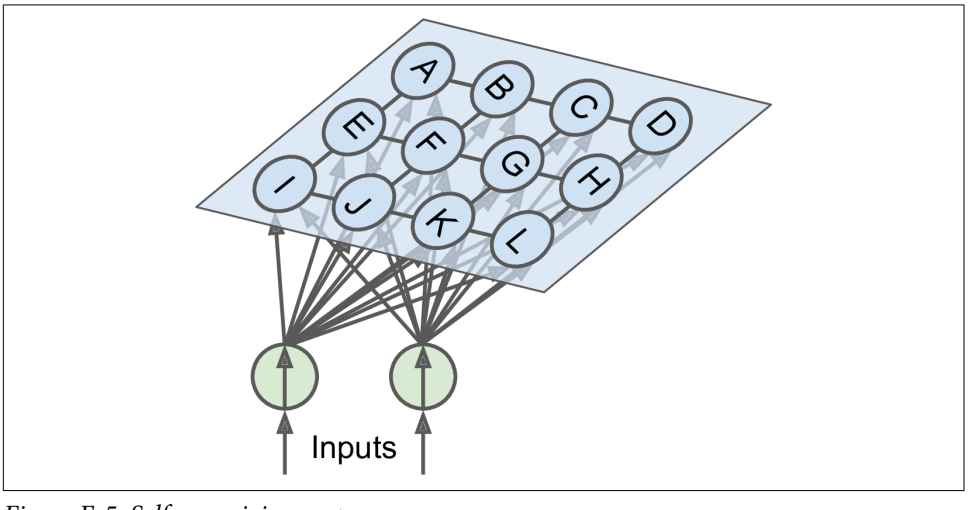



*Figure E-5. Self-organizing maps*

Once the network is trained, you can feed it a new instance and this will activate only one neuron (i.e., hence one point on the map): the neuron whose weight vector is closest to the input vector. In general, instances that are nearby in the original input space will activate neurons that are nearby on the map. This makes SOMs useful for visualization (in particular, you can easily identify clusters on the map), but also for applications like speech recognition. For example, if each instance represents the audio recording of a person pronouncing a vowel, then different pronunciations of the vowel "a" will activate neurons in the same area of the map, while instances of the vowel "e" will activate neurons in another area, and intermediate sounds will generally activate intermediate neurons on the map.



One important difference with the other dimensionality reduction techniques discussed in Chapter 8 is that all instances get mapped to a discrete number of points in the low-dimensional space (one point per neuron). When there are very few neurons, this technique is better described as clustering rather than dimensionality reduction.

The training algorithm is unsupervised. It works by having all the neurons compete against each other. First, all the weights are initialized randomly. Then a training instance is picked randomly and fed to the network. All neurons compute the distance between their weight vector and the input vector (this is very different from the artificial neurons we have seen so far). The neuron that measures the smallest distance wins and tweaks its weight vector to be even slightly closer to the input vector, making it more likely to win future competitions for other inputs similar to this one. It also recruits its neighboring neurons, and they too update their weight vector to be slightly closer to the input vector (but they don't update their weights as much as the winner neuron). Then the algorithm picks another training instance and repeats the process, again and again. This algorithm tends to make nearby neurons gradually specialize in similar inputs.[4]

4 You can imagine a class of young children with roughly similar skills. One child happens to be slightly better at basketball. This motivates her to practice more, especially with her friends. After a while, this group of friends gets so good at basketball that other kids cannot compete. But that's okay, because the other kids specialize in other topics. After a while, the class is full of little specialized groups.

# Index

## Symbols


__call__(), 390
ε-greedy policy, 465
ε-insensitive, 157
χ 2 test (see chi square test)
ℓ 0 norm, 39
ℓ 1 and ℓ 2 regularization, 307-308
ℓ 1 norm, 39, 132, 141, 304, 307
ℓ 2 norm, 39, 130-132, 141, 144, 307, 311
ℓ k norm, 39
ℓ ∞ norm, 39


## A


accuracy, 4, 85-86
actions, evaluating, 453-454
activation functions, 264-266
active constraints, 510
actual class, 87
AdaBoost, 194-197
Adagrad, 300-301
Adam optimization, 302-304, 304
adaptive learning rate, 301
adaptive moment optimization, 302
agents, 444
AlexNet architecture, 371-372
algorithms
    preparing data for, 60-69
AlphaGo, 14, 255, 443, 459
Anaconda, 41
anomaly detection, 12
Apple's Siri, 255
apply_gradients(), 288, 456
area under the curve (AUC), 94
array_split(), 219
artificial neural networks (ANNs), 255-276
    Boltzmann Machines, 522-524
    deep belief networks (DBNs), 525-527
    evolution of, 256
    Hopfield Networks, 521-522
    hyperparameter fine-tuning, 272-275
    overview, 255-257
    Perceptrons, 259-266
    self-organizing maps, 527-529
    training a DNN with TensorFlow, 267-272
artificial neuron, 258
    (see also artificial neural network (ANN))
assign(), 239
association rule learning, 12
associative memory networks, 521
assumptions, checking, 40
asynchronous updates, 352-353
asynchrous communication, 333-339
atrous_conv2d(), 380
attention mechanism, 414
attributes, 9, 45-49
    (see also data structure)
    combinations of, 59-60
    preprocessed, 48
    target, 48
autodiff, 240-241, 513-519
    forward-mode, 516-518
    manual differentiation, 513
    numerical differentiation, 515
    reverse-mode, 518-519
    symbolic differentiation, 514-515
autoencoders, 417-442
    adversarial, 440
    contractive, 439

denoising, 430-432
efficient data representations, 418
generative stochastic network (GSN), 440
overcomplete, 430
PCA with undercomplete linear autoencoder, 419
reconstructions, 419
sparse, 432-434
stacked, 421-430
stacked convolutional, 440
undercomplete, 419
variational, 435-439
visualizing features, 427-428
winner-take-all (WTA), 440
automatic differentiating, 233
autonomous driving systems, 383
Average Absolute Deviation, 39
average pooling layer, 369
avg_pool(), 369


## B


backpropagation, 263-264, 277, 295, 428
backpropagation through time (BPTT), 393
bagging and pasting, 187-190
out-of-bag evaluation, 189-190
in Scikit-Learn, 188-189
bandwidth saturation, 353-355
BasicLSTMCell, 405
BasicRNNCell, 402-402
Batch Gradient Descent, 116-119, 132
batch learning, 14-15
Batch Normalization, 284-288, 378
operation summary, 284
with TensorFlow, 286-288
batch(), 345
batch_join(), 345
batch_normalization(), 286-287
Bellman Optimality Equation, 461
between-graph replication, 348
bias neurons, 260
bias term, 108
bias/variance tradeoff, 128
biases, 269
binary classifiers, 84, 136
biological neurons, 256-258
black box models, 172
blending, 202-205
Boltzmann Machines, 522-524
(see also restricted Boltzman machines (RBMs))
boosting, 193-202
AdaBoost, 194-197
Gradient Boosting, 197-202
bootstrap aggregation (see bagging)
bootstrapping, 74, 187, 448, 475
bottleneck layers, 374
brew, 204


## C


Caffe model zoo, 294
CART (Classification and Regression Tree) algorithm, 172-173, 178
categorical attributes, 63
CategoricalEncoder, 68
cell wrapper, 397
chi square test, 176
classification versus regression, 8, 103
classifiers
binary, 84
error analysis, 98-101
evaluating, 98
MNIST dataset, 81-83
multiclass, 95-98
multilabel, 102-103
multioutput, 103-104
performance measures, 84-95
precision of, 87
voting, 183-186
clip_by_value(), 288
closed-form equation, 107, 130, 138
cluster specification, 328
clustering algorithms, 10
clusters, 327
coding space, 436
codings, 417
complementary slackness condition, 510
components_, 216
computational complexity, 112, 156, 174
compute_gradients(), 288, 455
concat(), 373
config.gpu_options, 322
ConfigProto, 321
confusion matrix, 86-88, 98-101
connectionism, 262
constrained optimization, 160, 509
Contrastive Divergence, 525
control dependencies, 327

conv1d(), 380
conv2d_transpose(), 380
conv3d(), 380
convergence rate, 119
convex function, 115
convolution kernels, 361, 369, 374
convolutional neural networks (CNNs), 357-382
  architectures, 369-380
    AlexNet, 371-372
    GoogleNet, 373-376
    LeNet5, 370-371
    ResNet, 376-379
  convolutional layer, 359-367, 374, 380
    feature maps, 362-364
    filters, 361
    memory requirement, 366-367
  evolution of, 358
  pooling layer, 367-369
  TensorFlow implementation, 364-366
Coordinator class, 342-344
correlation coefficient, 56-58
correlations, finding, 56-58
cost function, 20, 39
  in AdaBoost, 195
  in adagrad, 301
  in artificial neural networks, 267, 269-270
  in autodiff, 240
  in batch normalization, 287
  cross entropy, 371
  in Elastic Net, 134
  in Gradient Descent, 107, 113-114, 116, 119-121, 202, 277
  in Logistic Regression, 137-138
  in PG algorithms, 455
  in variational autoencoders, 436
  in Lasso Regression, 132-133
  in Linear Regression, 110, 115
  in Momentum optimization, 298-299
  in pretrained layers reuse, 297
  in ridge regression, 129-131
  in RNNs, 394, 398
  stale gradients and, 353
creative sequences, 400
credit assignment problem, 453-454
cross entropy, 142-143, 267, 434, 455
cross-validation, 30, 71-73, 85-86
CUDA library, 319
cuDNN library, 319
curse of dimensionality, 207-209
  (see also dimensionality reduction)
custom transformers, 65-66

## D

data, 30
  (see also test data; training data)
  creating workspace for, 40-43
  downloading, 43-45
  finding correlations in, 56-58
  making assumptions about, 30
  preparing for Machine Learning algorithms, 60-69
  test-set creation, 49-53
  working with real data, 33
data augmentation, 313-314
data cleaning, 61-63
data mining, 6
data parallelism, 351-355
  asynchronous updates, 352-353
  bandwidth saturation, 353-355
  synchronous updates, 352
  TensorFlow implementation, 355
data pipeline, 36
data snooping bias, 49
data structure, 45-49
data visualization, 53-56
DataFrame, 61
dataquest, xvi
decision boundaries, 139-141, 144, 172
decision function, 89, 158-159
Decision Stumps, 197
decision threshold, 89
Decision Trees, 70-72, 169-181, 183
  binary trees, 172
  class probability estimates, 173
  computational complexity, 174
  decision boundaries, 172
  GINI impurity, 174
  instability with, 179-180
  numbers of children, 172
  predictions, 171-173
  Random Forests (see Random Forests)
  regression tasks, 177-178
  regularization hyperparameters, 175-176
  training and visualizing, 169-171
decoder, 418
deconvolutional layer, 380
deep autoencoders (see stacked autoencoders)

deep belief networks (DBNs), 13, 525-527
Deep Learning, 443
  (see also Reinforcement Learning; TensorFlow)
  about, xiii, xvi
  libraries, 232-233
deep neural networks (DNNs), 263, 277-316
  (see also Multi-Layer Perceptrons (MLP))
  faster optimizers for, 297-306
  regularization, 307-314
  reusing pretrained layers, 289-297
  training guidelines overview, 314
  training with TensorFlow, 267-272
  training with TF.Learn, 266
  unstable gradients, 278
  vanishing and exploding gradients, 277-288
Deep Q-Learning, 466-475
  Ms. Pac Man example, 467-475
deep Q-network, 466
deep RNNs, 401-405
  applying dropout, 403
  distributing across multiple GPUs, 402
  long sequence difficulties, 404
  truncated backpropagation through time, 404
DeepMind, 14, 255, 443, 466
degrees of freedom, 27, 128
denoising autoencoders, 430-432
dense(), 269, 423
depth concat layer, 373
depth radius, 372
depthwise_conv2d(), 380
dequeue(), 336
dequeue_many(), 336, 338
dequeue_up_to(), 337-338
dequeuing data, 335
describe(), 46
device blocks, 331
device(), 323
dimensionality reduction, 12, 207-227, 417
  approaches to
    Manifold Learning, 212
    projection, 209-211
  choosing the right number of dimensions, 217
  curse of dimensionality, 207-209
  and data visualization, 207
  Isomap, 226
  LLE (Locally Linear Embedding), 223-225
  Multidimensional Scaling, 225-226
  PCA (Principal Component Analysis), 213-220
  t-Distributed Stochastic Neighbor Embedding (t-SNE), 226
discount rate, 453
distributed computing, 231
distributed sessions, 332-333
DNNClassifier, 266
drop(), 60
dropconnect, 311
dropna(), 61
dropout, 274, 403
dropout rate, 309
dropout(), 310
DRY (Don't Repeat Yourself), 249
Dual Averaging, 304
dual numbers, 516
dual problem, 162
duality, 509
dying ReLUs, 281
dynamic placements, 324
dynamic placer, 322
Dynamic Programming, 462
dynamic unrolling through time, 391
dynamic_rnn(), 391, 402, 413

## E

early stopping, 135-136, 200, 274, 307
Elastic Net, 134
embedded device blocks, 331
Embedded Reber grammars, 415
embeddings, 410-412
embedding_lookup(), 411
encoder, 418
Encoder–Decoder, 387
end-of-sequence (EOS) token, 393
energy functions, 522
enqueuing data, 334
Ensemble Learning, 72, 76, 183-205
  bagging and pasting, 187-190
  boosting, 193-202
  in-graph versus between-graph replication, 347-349
  Random Forests, 191-193
    (see also Random Forests)
  random patches and random subspaces, 190
  stacking, 202-204
entropy impurity measure, 174

environments, in reinforcement learning, 444-453, 465, 470
episodes (in RL), 450, 454-455, 457-458, 475
epochs, 120
ε-insensitive, 157
equality contraints, 510
error analysis, 98-101
estimators, 62
Euclidian norm, 39
eval(), 242
evaluating models, 29-31
explained variance, 217
explained variance ratio, 216
exploding gradients, 278
  (see also gradients, vanishing and exploding)
exploration policies, 465
exponential decay, 287
exponential linear unit (ELU), 282-283
exponential scheduling, 306
Extra-Trees, 192

## F

F-1 score, 88-89
face-recognition, 102
fake X server, 449
false positive rate (FPR), 93-95
fan-in, 279, 281
fan-out, 279, 281
feature detection, 417
feature engineering, 25
feature extraction, 12
feature importance, 192-193
feature maps, 222, 361-364, 378
feature scaling, 66
feature selection, 26, 75, 132, 193, 505
feature space, 220, 222
feature vector, 39, 109, 158, 239
features, 9
FeatureUnion, 69
feedforward neural network (FNN), 265
feed_dict, 242
FIFOQueue, 334, 337
fillna(), 61
first-in first-out (FIFO) queues, 334
first-order partial derivatives (Jacobians), 304
fit(), 61, 68, 219
fitness function, 20
fit_inverse_transform=, 223
fit_transform(), 62, 68
folds, 71, 83, 85-86
Follow The Regularized Leader (FTRL), 304
forget gate, 406
forward-mode autodiff, 516-518
framing a problem, 35-37
frozen layers, 292-294
functools.partial(), 287, 422, 437

## G

game play (see reinforcement learning)
gamma value, 154
gate controllers, 407
Gaussian distribution, 435, 438
Gaussian RBF, 153
Gaussian RBF kernel, 154-155, 165
generalization error, 29
generalized Lagrangian, 510-511
generative autoencoders, 435
generative models, 417, 524
genetic algorithms, 446
geodesic distance, 226
get_variable(), 251-252
GINI impurity, 171, 174
global average pooling, 376
global_step, 472
global_variables_initializer(), 235
Glorot initialization, 278-281
Google, 232
Google Images, 255
Google Photos, 13
GoogleNet architecture, 373-376
gpu_options.per_process_gpu_memory_fraction, 321
gradient ascent, 447
Gradient Boosted Regression Trees (GBRT), 197
Gradient Boosting, 197-202
Gradient Descent (GD), 107, 113-123, 166, 277, 297, 300
  algorithm comparisons, 121-123
  automatically computing gradients, 240-241
  Batch GD, 116-119, 132
  defining, 113
  local minimum versus global minimum, 114
  manually computing gradients, 239
  Mini-batch GD, 121-123, 241-243
  optimizer, 241
  Stochastic GD, 119-121, 150

with TensorFlow, 239-241
Gradient Tree Boosting, 197
GradientDescentOptimizer, 270
gradients(), 240
gradients, vanishing and exploding, 277-288, 404
Batch Normalization, 284-288
Glorot and He initialization, 278-281
gradient clipping, 288
nonsaturating activation functions, 281-283
graphviz, 170
greedy algorithm, 174
grid search, 73-75, 153
group(), 471
GRU (Gated Recurrent Unit) cell, 408-410

## H

hailstone sequence, 418
hard margin classification, 148-149
hard voting classifiers, 183-186
harmonic mean, 88
He initialization, 278-281
Heaviside step function, 259
Hebb's rule, 260, 522
Hebbian learning, 261
hidden layers, 263
hierarchical clustering, 10
hinge loss function, 166
histograms, 47-49
hold-out sets, 202
(see also blenders)
Hopfield Networks, 521-522
hyperbolic tangent (htan activation function), 264, 274, 278, 280, 385
hyperparameters, 28, 66, 73-75, 77, 113, 153, 156, 272
(see also neural network hyperparameters)
hyperplane, 159, 212-213, 215, 226
hypothesis, 39
manifold, 212
hypothesis boosting (see boosting)
hypothesis function, 109
hypothesis, null, 176

## I

identity matrix, 130, 162
ILSVRC ImageNet challenge, 369
image classification, 369
impurity measures, 171, 174
in-graph replication, 347
inception modules, 373
Inception-v4, 379
incremental learning, 16, 219
inequality constraints, 510
inference, 22, 315, 367, 413
info(), 45
information gain, 175
information theory, 174
init node, 243
input gate, 406
input neurons, 260
input_keep_prob, 404
instance-based learning, 17, 21
InteractiveSession, 235
intercept term, 108
Internal Covariate Shift problem, 284
inter_op_parallelism_threads, 326
intra_op_parallelism_threads, 326
inverse_transform(), 223
in_top_k(), 270
irreducible error, 129
isolated environment, 41-42
Isomap, 226

## J

jobs, 327
join(), 329, 343
Jupyter, 40, 42, 47

## K

K-fold cross-validation, 71-73, 85
k-Nearest Neighbors, 21, 102
Karush–Kuhn–Tucker (KKT) conditions, 510
keep probability, 310
Keras, 233
Kernel PCA (kPCA), 220-223
kernel trick, 152, 154, 163-166, 220
kernelized SVM, 163-166
kernels, 152-155, 325
Kullback–Leibler divergence, 143, 432

## L

l1_l2_regularizer(), 308
labels, 8, 37
Lagrange function, 510-511
Lagrange multiplier, 509
landmarks, 153-154

large margin classification, 147-148
Lasso Regression, 132-134
latent loss, 436
latent space, 436
law of large numbers, 185
leaky ReLU, 281
learning rate, 16, 113, 117-120
learning rate scheduling, 120, 305-306
LeNet-5 architecture, 359, 370-371
Levenshtein distance, 155
liblinear library, 156
libsvm library, 156
Linear Discriminant Analysis (LDA), 226
linear models
  early stopping, 135-136
  Elastic Net, 134
  Lasso Regression, 132-134
  Linear Regression (see Linear Regression)
  regression (see Linear Regression)
  Ridge Regression, 129-131, 134
  SVM, 147-150
Linear Regression, 20, 69, 107-123, 134
  computational complexity, 112
  Gradient Descent in, 113-123
  learning curves in, 125-129
  Normal Equation, 110-112
  regularizing models (see regularization)
  using Stochastic Gradient Descent (SGD), 121
  with TensorFlow, 237-238
linear SVM classification, 147-150
linear threshold units (LTUs), 259
Lipschitz continuous, 115
LLE (Locally Linear Embedding), 223-225
load_sample_images(), 364
local receptive field, 358
local response normalization, 372
local sessions, 332
log loss, 138
logging placements, 324-324
logistic function, 137
Logistic Regression, 9, 136-144
  decision boundaries, 139-141
  estimating probablities, 136-137
  Softmax Regression model, 141-144
  training and cost function, 137-138
log_device_placement, 324
LSTM (Long Short-Term Memory) cell, 405-410

## M

machine control (see reinforcement learning)
Machine Learning
  large-scale projects (see TensorFlow)
  notations, 38-39
  process example, 33-79
  project checklist, 35, 503-508
  resources on, xvi-xvii
  uses for, xiii-xiv
Machine Learning basics
  attributes, 9
  challenges, 22-29
    algorithm problems, 26-28
    training data problems, 25
  definition, 4
  features, 9
  overview, 3
  reasons for using, 4-7
  spam filter example, 4-6
  summary, 28
  testing and validating, 29-31
  types of systems, 7-22
    batch and online learning, 14-17
    instance-based versus model-based learning, 17-22
    supervised/unsupervised learning, 8-14
  workflow example, 18-22
machine translation (see natural language processing (NLP))
make(), 448
Manhattan norm, 39
manifold assumption/hypothesis, 212
Manifold Learning, 212, 223
  (see also LLE (Locally Linear Embedding)
MapReduce, 37
margin violations, 149
Markov chains, 459
Markov decision processes, 459-463
master service, 329
Matplotlib, 40, 47, 93, 99
max margin learning, 297
max pooling layer, 368
max-norm regularization, 311-313
max_norm(), 312
max_norm_regularizer(), 312
max_pool(), 369
Mean Absolute Error (MAE), 39-40
mean coding, 435
Mean Square Error (MSE), 109, 239, 432

measure of similarity, 17
memmap, 219
memory cells, 350, 386
Mercer's theorem, 165
meta learner (see blending)
min-max scaling, 66
Mini-batch Gradient Descent, 121-123, 138, 241-243
mini-batches, 15
minimize(), 288, 293, 455
min_after_dequeue, 337
MNIST dataset, 81-83
model parallelism, 349-351
model parameters, 116, 118, 135, 158, 161, 236, 270, 394
  defining, 19
model selection, 19
model zoos, 294
model-based learning, 18-22
models
  analyzing, 76-77
  evaluating on test set, 77-77
moments, 302
Momentum optimization, 297-298
Monte Carlo tree search, 459
Multi-Layer Perceptrons (MLP), 255, 262-265, 452
  training with TF.Learn, 266
multiclass classifiers, 95-98
Multidimensional Scaling (MDS), 225
multilabel classifiers, 102-103
Multinomial Logistic Regression (see Softmax Regression)
multinomial(), 453
multioutput classifiers, 103-104
MultiRNNCell, 402
multithreaded readers, 342-344
multivariate regression, 37

## N

naive Bayes classifiers, 96
name scopes, 247
natural language processing (NLP), 383, 410-415
  encoder-decoder network for machine translation, 412-415
  TensorFlow tutorials, 410, 413
  word embeddings, 410-412
Nesterov Accelerated Gradient (NAG), 299-299
Nesterov momentum optimization, 299-299
network topology, 272
neural network hyperparameters, 272-275
  activation functions, 274
  neurons per hidden layer, 274
  number of hidden layers, 273-274
neural network policies, 451-453
neurons
  biological, 256-258
  logical computations with, 258
neuron_layer(), 269
next_batch(), 271
No Free Lunch theorem, 30
node edges, 246
nonlinear dimensionality reduction (NLDR), 223
  (see also Kernel PCA; LLE (Locally Linear Embedding))
nonlinear SVM classification, 151-156
  computational complexity, 156
  Gaussian RBF kernel, 154-155
  with polynomial features, 151-152
  polynomial kernel, 152-153
  similarity features, adding, 153-154
nonparametric models, 175
nonresponse bias, 25
nonsaturating activation functions, 281-283
Normal Equation, 110-112
normalization, 67
normalized exponential, 141
norms, 39
notations, 38-39
NP-Complete problems, 174
null hypothesis, 176
numerical differentiation, 515
NumPy, 40
NumPy arrays, 64
NVidia Compute Capability, 318
nvidia-smi, 322
n_components, 217

## O

observation space, 452
off-policy algorithm, 465
offline learning, 14
one-hot encoding, 64
one-versus-all (OvA) strategy, 96, 144, 167
one-versus-one (OvO) strategy, 96
online learning, 15-17

online SVMs, 166-167
OpenAI Gym, 447-450
operation_timeout_in_ms, 349
Optical Character Recognition (OCR), 3
optimal state value, 461
optimizers, 297-306
  AdaGrad, 300-301
  Adam optimization, 302-304, 304
  Gradient Descent (see Gradient Descent optimizer)
  learning rate scheduling, 305-306
  Momentum optimization, 297-298
  Nesterov Accelerated Gradient (NAG), 299-299
  RMSProp, 302
out-of-bag evaluation, 189-190
out-of-core learning, 16
out-of-memory (OOM) errors, 391
out-of-sample error, 29
OutOfRangeError, 341, 343
output gate, 406
output layer, 263
OutputProjectionWrapper, 397-398
output_keep_prob, 404
overcomplete autoencoder, 430
overfitting, 26-28, 49, 149, 155, 175, 178, 274
  avoiding through regularization, 307-314

## P

p-value, 176
PaddingFIFOQueue, 338
Pandas, 40, 44
  scatter_matrix, 57-58
parallel distributed computing, 317-356
  data parallelism, 351-355
  in-graph versus between-graph replication, 347-349
  model parallelism, 349-351
  multiple devices across multiple servers, 327-346
    asynchronous communication using queues, 333-339
    loading training data, 339-346
    master and worker services, 329
    opening a session, 329
    pinning operations across tasks, 330
    sharding variables, 331
    sharing state across sessions, 332-333
  multiple devices on a single machine, 318-327
    control dependencies, 327
    installation, 318-320
    managing the GPU RAM, 321-322
    parallel execution, 325-326
    placing operations on devices, 322-325
  one neural network per device, 346-347
parameter efficiency, 273
parameter matrix, 142
parameter server (ps), 328
parameter space, 116
parameter vector, 109, 113, 137, 142
parametric models, 175
partial derivative, 116
partial_fit(), 219
Pearson's r, 56
peephole connections, 408
penalties (see rewards, in RL)
percentiles, 47
Perceptron convergence theorem, 261
Perceptrons, 259-266
  versus Logistic Regression, 262
  training, 260-261
performance measures, 37-40
  confusion matrix, 86-88
  cross-validation, 85-86
  precision and recall, 88-92
  ROC (receiver operating characteristic) curve, 93-95
performance scheduling, 305
permutation(), 50
PG algorithms, 454
photo-hosting services, 13
pinning operations, 330
pip, 41
Pipeline constructor, 67-69
pipelines, 36
placeholder nodes, 241
placers (see simple placer; dynamic placer)
policy, 446
policy gradients, 447 (see PG algorithms)
policy space, 446
polynomial features, adding, 151-152
polynomial kernel, 152-153, 164
Polynomial Regression, 108, 123-125
  learning curves in, 125-129
pooling kernel, 368
pooling layer, 367-369

power scheduling, 306
precision, 87
precision and recall, 88-92
  F-1 score, 88-89
  precision/recall (PR) curve, 94
  precision/recall tradeoff, 89-92
predetermined piecewise constant learning rate, 305
predict(), 62
predicted class, 87
predictions, 86-88, 158-159, 171-173
predictors, 8, 62
preloading training data, 339
PReLU (parametric leaky ReLU), 281
preprocessed attributes, 48
pretrained layers reuse, 289-297
  auxiliary task, 296-297
  caching frozen layers, 293
  freezing lower layers, 292
  model zoos, 294
  other frameworks, 291
  TensorFlow model, 289-291
  unsupervised pretraining, 295-295
  upper layers, 294
Pretty Tensor, 233
primal problem, 162
principal component, 214
Principal Component Analysis (PCA), 213-220
  explained variance ratios, 216
  finding principal components, 214-215
  for compression, 218-219
  Incremental PCA, 219-220
  Kernel PCA (kPCA), 220-223
  projecting down to d dimensions, 215
  Randomized PCA, 220
  Scikit Learn for, 216
  variance, preserving, 213-214
probabilistic autoencoders, 435
probabilities, estimating, 136-137, 173
producer functions, 345
projection, 209-211
propositional logic, 256
pruning, 176, 515
Python
  isolated environment in, 41-42
  notebooks in, 42-43
  pickle, 73
  pip, 41

## Q

Q-Learning algorithm, 464-475
  approximate Q-Learning, 466
  deep Q-Learning, 466-475
Q-Value Iteration Algorithm, 462
Q-Values, 462
Quadratic Programming (QP) Problems, 161-162
quantizing, 355
queries per second (QPS), 347
QueueRunner, 342-344
queues, 333-339
  closing, 337
  dequeuing data, 335
  enqueuing data, 334
  first-in first-out (FIFO), 334
  of tuples, 336
  PaddingFIFOQueue, 338
  RandomShuffleQueue, 337
q_network(), 470

## R

Radial Basis Function (RBF), 153
Random Forests, 72-73, 96, 169, 180, 183, 191-193
  Extra-Trees, 192
  feature importance, 192-193
random initialization, 113, 118, 120, 278
Random Patches and Random Subspaces, 190
randomized leaky ReLU (RReLU), 281
Randomized PCA, 220
randomized search, 75, 272
RandomShuffleQueue, 337, 341
random_uniform(), 239
reader operations, 340
recall, 87
recognition network, 418
reconstruction error, 218
reconstruction loss, 419, 434, 436
reconstruction pre-image, 222
reconstructions, 419
recurrent neural networks (RNNs), 383-415
  deep RNNs, 401-405
  exploration policies, 465
  GRU cell, 408-410
  input and output sequences, 387-388
  LSTM cell, 405-410
  natural language processing (NLP), 410-415
  in TensorFlow, 388-393

dynamic unrolling through time, 391
static unrolling through time, 389-391
variable length input sequences, 392
variable length output sequences, 393
training, 393-401
backpropagation through time (BPTT), 393
creative sequences, 400
sequence classifiers, 394-396
time series predictions, 396-400
recurrent neurons, 384-388
memory cells, 386
reduce_mean(), 270
reduce_sum(), 434-434, 437, 472
regression, 8
Decision Trees, 177-178
regression models
linear, 69
regression versus classification, 103
regularization, 27-28, 30, 129-136
data augmentation, 313-314
Decision Trees, 175-176
dropout, 309-311
early stopping, 135-136, 307
Elastic Net, 134
Lasso Regression, 132-134
max-norm, 311-313
Ridge Regression, 129-131
shrinkage, 199
ℓ 1 and ℓ 2 regularization, 307-308
REINFORCE algorithms, 454
Reinforcement Learning (RL), 13-14, 443-476
actions, 453-454
credit assignment problem, 453-454
discount rate, 453
examples of, 444
Markov decision processes, 459-463
neural network policies, 451-453
OpenAI gym, 447-450
PG algorithms, 454-459
policy search, 446-447
Q-Learning algorithm, 464-475
rewards, learning to optimize, 444-445
Temporal Difference (TD) Learning, 463-464
ReLU (rectified linear units), 248-250
ReLU activation, 378
ReLU function, 264, 274, 280-283
relu(z), 269
render(), 448
replica_device_setter(), 331
request_stop(), 343
reset(), 448
reset_default_graph(), 236
reshape(), 400
residual errors, 197-198
residual learning, 376
residual network (ResNet), 294, 376-379
residual units, 377
ResNet, 376-379
resource containers, 332-333
restore(), 243
restricted Boltzmann machines (RBMs), 13, 295, 524
reuse_variables(), 251
reverse-mode autodiff, 518-519
rewards, in RL, 444-445
rgb_array, 449
Ridge Regression, 129-131, 134
RMSProp, 302
ROC (receiver operating characteristic) curve, 93-95
Root Mean Square Error (RMSE), 37-40, 109
RReLU (randomized leaky ReLU), 281
run(), 235, 349

## S

Sampled Softmax, 414
sampling bias, 24-25, 51
sampling noise, 24
save(), 243
Saver node, 243
Scikit Flow, 233
Scikit-Learn, 40
about, xiv
bagging and pasting in, 188-189
CART algorithm, 172-173, 178
cross-validation, 71-73
design principles, 62-63
imputer, 61-63
LinearSVR class, 158
MinMaxScaler, 67
min_ and max_ hyperparameters, 175
PCA implementation, 216
Perceptron class, 261
Pipeline constructor, 67-69, 151
Randomized PCA, 220
Ridge Regression with, 131

SAMME, 197
SGDClassifier, 84, 89-90, 96
SGDRegressor, 121
sklearn.base.BaseEstimator, 66, 68, 86
sklearn.base.clone(), 85, 135
sklearn.base.TransformerMixin, 65, 68
sklearn.datasets.fetch_california_housing(), 238
sklearn.datasets.fetch_mldata(), 81
sklearn.datasets.load_iris(), 139, 150, 169, 192, 261
sklearn.datasets.load_sample_images(), 364-365
sklearn.datasets.make_moons(), 151, 180
sklearn.decomposition.IncrementalPCA, 219
sklearn.decomposition.KernelPCA, 220-221, 223
sklearn.decomposition.PCA, 216
sklearn.ensemble.AdaBoostClassifier, 197
sklearn.ensemble.BaggingClassifier, 188-191
sklearn.ensemble.GradientBoostingRegressor, 198, 200-201
sklearn.ensemble.RandomForestClassifier, 94, 97, 186
sklearn.ensemble.RandomForestRegressor, 72, 73-76, 191-192, 198
sklearn.ensemble.VotingClassifier, 186
sklearn.externals.joblib, 73
sklearn.linear_model.ElasticNet, 134
sklearn.linear_model.Lasso, 134
sklearn.linear_model.LinearRegression, 20-21, 62, 69, 112, 122, 124, 126-127
sklearn.linear_model.LogisticRegression, 139, 141, 144, 186, 221
sklearn.linear_model.Perceptron, 261
sklearn.linear_model.Ridge, 131
sklearn.linear_model.SGDClassifier, 84
sklearn.linear_model.SGDRegressor, 121-122, 131, 134-135
sklearn.manifold.LocallyLinearEmbedding, 223-224
sklearn.metrics.accuracy_score(), 186, 190, 266
sklearn.metrics.confusion_matrix(), 87, 98
sklearn.metrics.f1_score(), 89, 102
sklearn.metrics.mean_squared_error(), 70-70, 77, 126, 135, 200-201, 223
sklearn.metrics.precision_recall_curve(), 90
sklearn.metrics.precision_score(), 88, 92
sklearn.metrics.recall_score(), 88, 92
sklearn.metrics.roc_auc_score(), 94-95
sklearn.metrics.roc_curve(), 93-94
sklearn.model_selection.cross_val_predict(), 86, 90, 94, 98, 102
sklearn.model_selection.cross_val_score(), 71-72, 85-86
sklearn.model_selection.GridSearchCV, 73-75, 79, 98, 181, 221
sklearn.model_selection.StratifiedKFold, 85
sklearn.model_selection.StratifiedShuffleSplit, 52
sklearn.model_selection.train_test_split(), 51, 71, 126, 180, 200
sklearn.multiclass.OneVsOneClassifier, 97
sklearn.neighbors.KNeighborsClassifier, 102, 104
sklearn.neighbors.KNeighborsRegressor, 22
sklearn.pipeline.FeatureUnion, 68
sklearn.pipeline.Pipeline, 67, 127, 150-151, 221
sklearn.preprocessing.CategoricalEncoder, 64, 68
sklearn.preprocessing.Imputer, 61, 69
sklearn.preprocessing.OneHotEncoder, 64
sklearn.preprocessing.PolynomialFeatures, 124-125, 127, 130, 151
sklearn.preprocessing.StandardScaler, 67-68, 98, 116, 130, 148, 150-152, 154, 239, 266
sklearn.svm.LinearSVC, 149-151, 155-156, 158, 167
sklearn.svm.LinearSVR, 157-158
sklearn.svm.SVC, 150, 152, 154-156, 158, 167, 186
sklearn.svm.SVR, 79, 158
sklearn.tree.DecisionTreeClassifier, 175, 181, 188-189, 191, 197
sklearn.tree.DecisionTreeRegressor, 70, 169, 177, 197-198
sklearn.tree.export_graphviz(), 170
StandardScaler, 116, 239, 266
SVM classification classes, 156
TF.Learn, 233
user guide, xvi
score(), 62
search space, 75, 272
second-order partial derivatives (Hessians), 304

self-organizing maps (SOMs), 527-529
semantic hashing, 441
semisupervised learning, 13
sensitivity, 87, 93
sentiment analysis, 383
separable_conv2d(), 380
sequences, 383
sequence_length, 392-393, 413
Shannon's information theory, 174
shortcut connections, 376
show(), 47
show_graph(), 247
shrinkage, 199
shuffle_batch(), 345
shuffle_batch_join(), 345
sigmoid function, 137
sigmoid_cross_entropy_with_logits(), 434
similarity function, 153-154
simulated annealing, 120
simulated environments, 448
  (see also OpenAI Gym)
Singular Value Decomposition (SVD), 215
skewed datasets, 86
skip connections, 314, 376
slack variable, 160
smoothing terms, 285, 301, 303, 437
soft margin classification, 148-150
soft placements, 325
soft voting, 186
softmax function, 141, 265, 267
Softmax Regression, 141-144
source ops, 238, 326
spam filters, 3-6, 8
sparse autoencoders, 432-434
sparse matrix, 64
sparse models, 132, 304
sparse_softmax_cross_entropy_with_logits(), 270
sparsity loss, 432
specificity, 93
speech recognition, 6
spurious patterns, 522
stack(), 390
stacked autoencoders, 421-430
  TensorFlow implementation, 422
  training one-at-a-time, 424-426
  tying weights, 423-424
  unsupervised pretraining with, 428-430
  visualizing the reconstructions, 427-427
stacked denoising autoencoders, 428, 430
stacked denoising encoders, 430
stacked generalization (see stacking)
stacking, 202-204
stale gradients, 352
standard correlation coefficient, 56
standardization, 66
StandardScaler, 68, 239, 266
state-action values, 462
states tensor, 392
state_is_tuple, 403, 405
state_keep_prob, 404
static unrolling through time, 389-391
static_rnn(), 389-391, 413
stationary point, 509-511
statistical mode, 187
statistical significance, 176
stemming, 105
step functions, 259
step(), 450
Stochastic Gradient Boosting, 201
Stochastic Gradient Descent (SGD), 119-121, 150, 262
  training, 138
Stochastic Gradient Descent (SGD) classifier, 84, 131
stochastic neurons, 522
stochastic policy, 446
stratified sampling, 51-53, 85
stride, 361
string kernels, 155
string_input_producer(), 345
strong learners, 184
subderivatives, 166
subgradient vector, 133
subsample, 201, 367
supervised learning, 8-9
Support Vector Machines (SVMs), 96, 147-168
  decision function and predictions, 158-159
  dual problem, 509-511
  kernelized SVM, 163-166
  linear classification, 147-150
  mechanics of, 158-167
  nonlinear classification, 151-156
  online SVMs, 166-167
  Quadratic Programming (QP) problems, 161-162
  SVM regression, 156-167
  the dual problem, 162

training objective, 159-161
support vectors, 148
svd(), 215
symbolic differentiation, 240, 514-515
synchronous updates, 352

## T

t-Distributed Stochastic Neighbor Embedding (t-SNE), 226
tail heavy, 49
target attributes, 48
target_weights, 414
tasks, 327
Temporal Difference (TD) Learning, 463-464
tensor processing units (TPUs), 319
TensorBoard, 233
TensorFlow, 231-254
about, xiv
autodiff, 240-241, 513-519
Batch Normalization with, 286-288
construction phase, 236
control dependencies, 327
convenience functions, 345
convolutional layers, 380
convolutional neural networks and, 364-366
data parallelism and, 355
denoising autoencoders, 431-432
dropout with, 310
dynamic placer, 322
execution phase, 236
feeding data to the training algorithm, 241-243
Gradient Descent with, 239-241
graphs, managing, 236
initial graph creation and session run, 234-236
installation, 234
l1 and l2 regularization with, 307
learning schedules in, 306
Linear Regression with, 237-238
max pooling layer in, 368
max-norm regularization with, 311
model zoo, 294
modularity, 248-250
Momentum optimization in, 298
name scopes, 247
neural network policies, 452
NLP tutorials, 410, 413
node value lifecycle, 237
operations (ops), 237
optimizer, 241
overview, 231-233
parallel distributed computing (see parallel distributed computing with TensorFlow)
Python API
construction, 267-271
execution, 271
using the neural network, 272
queues (see queues)
reusing pretrained layers, 289-291
RNNs in, 388-393
(see also recurrent neural networks (RNNs))
saving and restoring models, 243-244
sharing variables, 250-253
simple placer, 322
sparse autoencoders with, 434
and stacked autoencoders, 422
TensorBoard, 244-247
tf.abs(), 307
tf.add(), 248, 307
tf.add_n(), 249-250, 252-253
tf.add_to_collection(), 312
tf.assign(), 239, 291, 311-312, 488
tf.bfloat16, 354
tf.bool, 310
tf.cast(), 270, 395
tf.clip_by_norm(), 311-312
tf.clip_by_value(), 288
tf.concat(), 316, 373, 452, 456
tf.ConfigProto, 321, 324-325, 349, 493
tf.constant(), 237-239, 323-324, 327, 329-330
tf.constant_initializer(), 251-253
tf.container(), 332-334, 355-356, 487
tf.contrib.layers.l1_regularizer(), 308, 312
tf.contrib.layers.l2_regularizer(), 308, 422-424
tf.contrib.layers.variance_scaling_initializer(), 280-281, 396, 422-424, 437, 452, 456, 470
tf.contrib.learn.DNNClassifier, 266
tf.contrib.learn.infer_real_valued_columns_from_input(), 266
tf.contrib.rnn.BasicLSTMCell, 405, 408
tf.contrib.rnn.BasicRNNCell, 389-391, 394, 396-397, 398, 401-403, 405
tf.contrib.rnn.DropoutWrapper, 403

tf.contrib.rnn.GRUCell, 410
tf.contrib.rnn.LSTMCell, 408
tf.contrib.rnn.MultiRNNCell, 401-403
tf.contrib.rnn.OutputProjectionWrapper, 397-398
tf.contrib.rnn.RNNCell, 402
tf.contrib.rnn.static_rnn(), 389-392, 413-415, 498-498
tf.contrib.slim module, 233, 381
tf.contrib.slim.nets module (nets), 381
tf.control_dependencies(), 327
tf.decode_csv(), 340, 344
tf.device(), 323-325, 330-331, 402-402
tf.exp(), 437-438
tf.FIFOQueue, 334, 336-337, 340, 344
tf.float32, 238, 488
tf.get_collection(), 291-292, 308, 312, 422, 470
tf.get_default_graph(), 236, 245
tf.get_default_session(), 235
tf.get_variable(), 251-253, 291, 308
tf.global_variables(), 290
tf.global_variables_initializer(), 235, 239
tf.gradients(), 240
tf.Graph, 234, 236, 244, 339, 347
tf.GraphKeys.GLOBAL_VARIABLES, 291-292
tf.GraphKeys.REGULARIZATION_LOSSES, 308, 422
tf.group(), 471
tf.int32, 325-336, 341, 392, 394, 411, 472
tf.int64, 267
tf.InteractiveSession, 235
tf.layers.batch_normalization(), 286-287
tf.layers.dense(), 269
TF.Learn, 266
tf.log(), 434, 437, 452, 456
tf.matmul(), 238-239, 248, 267, 388, 424, 426, 431, 434-434
tf.matrix_inverse(), 238
tf.maximum(), 248, 250-253, 283
tf.multinomial(), 452, 456
tf.name_scope(), 247, 250-251, 267, 269-270, 426-426
tf.nn.conv2d(), 364-365
tf.nn.dynamic_rnn(), 391-391, 394, 396, 400, 401-403, 413-415, 498-498
tf.nn.elu(), 283, 422-424, 437, 452, 456
tf.nn.embedding_lookup(), 411
tf.nn.in_top_k(), 270, 395
tf.nn.max_pool(), 368-369
tf.nn.relu(), 267, 396-397, 398, 470
tf.nn.sigmoid_cross_entropy_with_logits(), 434, 438, 455-456
tf.nn.sparse_softmax_cross_entropy_with_logits(), 269-270, 394
tf.one_hot(), 472
tf.PaddingFIFOQueue, 338
tf.placeholder(), 241-242, 488
tf.placeholder_with_default(), 431
tf.RandomShuffleQueue, 337, 341-342, 344-345
tf.random_normal(), 248, 388, 431, 437
tf.random_uniform(), 239, 243, 411, 488
tf.reduce_mean(), 239, 247, 269-271, 307, 394-395, 420, 422, 424, 426, 431, 434, 472
tf.reduce_sum(), 307, 434-434, 436-438, 472-472
tf.reset_default_graph(), 236
tf.reshape(), 400, 470
tf.RunOptions, 349
tf.Session, 235, 488
tf.shape(), 431, 437
tf.square(), 239, 247, 398, 420, 422, 424, 426, 431, 434, 437-438, 472
tf.stack(), 340, 344, 390
tf.string, 340, 344
tf.summary.FileWriter, 245-245
tf.summary.scalar(), 245
tf.tanh(), 388
tf.TextLineReader, 340, 344
tf.to_float(), 455-456
tf.train.AdamOptimizer, 303, 304, 394, 398, 420, 422-424, 426, 434, 438, 455-456, 472
tf.train.ClusterSpec, 328
tf.train.Coordinator, 342-344
tf.train.exponential_decay(), 306
tf.train.GradientDescentOptimizer, 241, 270, 288, 298, 304
tf.train.MomentumOptimizer, 241, 298-299, 306, 315, 355, 491-492
tf.train.QueueRunner, 342-345
tf.train.replica_device_setter(), 331-332
tf.train.RMSPropOptimizer, 302
tf.train.Saver, 243-244, 271, 381, 456, 472
tf.train.Server, 328

tf.train.start_queue_runners(), 345
tf.transpose(), 238-239, 390, 424
tf.truncated_normal(), 267
tf.unstack(), 390-391, 399, 498
tf.Variable, 234, 488
tf.variable_scope(), 251-253, 291, 332, 396, 470
tf.zeros(), 267, 388, 424
truncated backpropagation through time, 404
visualizing graph and training curves, 244-247
TensorFlow Serving, 347
tensorflow.contrib, 266
test set, 29, 49-53, 83
testing and validating, 29-31
text attributes, 63
TextLineReader, 340
TF-slim, 233
tf.layers.conv1d(), 380
tf.layers.conv2d(), 470
tf.layers.conv2d_transpose(), 380
tf.layers.conv3d(), 380
tf.layers.dense(), 280, 286
tf.layers.separable_conv2d(), 380
TF.Learn, 233, 266
tf.nn.atrous_conv2d(), 380
tf.nn.depthwise_conv2d(), 380
thermal equilibrium, 524
thread pools (inter-op/intra-op, in TensorFlow, 326
threshold variable, 250-253
Tikhonov regularization, 129
time series data, 383
toarray(), 64
tolerance hyperparameter, 156
training, 286-287
training data, 4
insufficient quantities, 22
irrelevant features, 25
loading, 339-346
nonrepresentative, 24
overfitting, 26-28
poor quality, 25
underfitting, 28
training instance, 4
training models, 20, 107-145
learning curves in, 125-129
Linear Regression, 107, 108-123
Logistic Regression, 136-144
overview, 107-108
Polynomial Regression, 108, 123-125
training objectives, 159-161
training set, 4, 29, 53, 60, 69-71
cost function of, 137-138
shuffling, 83
transfer learning, 289-297
(see also pretrained layers reuse)
transform(), 62, 68
transformation pipelines, 67-69
transformers, 62
transformers, custom, 65-66
transpose(), 390
true negative rate (TNR), 93
true positive rate (TPR), 87, 93
truncated backpropagation through time, 404
tuples, 336
tying weights, 423


## U


underfitting, 28, 70, 155
univariate regression, 37
unstack(), 390
unsupervised learning, 10-12
anomaly detection, 12
association rule learning, 10, 12
clustering, 10
dimensionality reduction algorithm, 12
visualization algorithms, 11
unsupervised pretraining, 295-295, 428-430
upsampling, 380
utility function, 20


## V


validation set, 30
Value Iteration, 461
value_counts(), 46
vanishing gradients, 278
(see also gradients, vanishing and exploding)
variables, sharing, 250-253
variable_scope(), 251-252
variance
bias/variance tradeoff, 128
variance preservation, 213-214
variance_scaling_initializer(), 280
variational autoencoders, 435-439
VGGNet, 379

visual cortex, 358
visualization, 244-247
visualization algorithms, 11-12
voice recognition, 357
voting classifiers, 183-186

## W

warmup phase, 353
weak learners, 184
weight-tying, 423
weights, 269
  freezing, 292
while_loop(), 391
white box models, 172
worker, 328
worker service, 329
worker_device, 331
workspace directory, 40-43

## X

Xavier initialization, 278-281

## Y

YouTube, 255

## Z

zero padding, 360, 365

## About the Author

**Aurélien Géron** is a Machine Learning consultant. A former Googler, he led the YouTube video classification team from 2013 to 2016. He was also a founder and CTO of Wifirst from 2002 to 2012, a leading Wireless ISP in France; and a founder and CTO of Polyconseil in 2001, the firm that now manages the electric car sharing service Autolib'.

Before this he worked as an engineer in a variety of domains: finance (JP Morgan and Société Générale), defense (Canada's DOD), and healthcare (blood transfusion). He published a few technical books (on C++, WiFi, and internet architectures), and was a Computer Science lecturer in a French engineering school.

A few fun facts: he taught his three children to count in binary with their fingers (up to 1023), he studied microbiology and evolutionary genetics before going into software engineering, and his parachute didn't open on the second jump.

## Colophon

The animal on the cover of *Hands-On Machine Learning with Scikit-Learn and TensorFlow* is the far eastern fire salamander (*Salamandra infraimmaculata*), an amphibian found in the Middle East. They have black skin featuring large yellow spots on their back and head. These spots are a warning coloration meant to keep predators at bay. Full-grown salamanders can be over a foot in length.

Far eastern fire salamanders live in subtropical shrubland and forests near rivers or other freshwater bodies. They spend most of their life on land, but lay their eggs in the water. They subsist mostly on a diet of insects, worms, and small crustaceans, but occasionally eat other salamanders. Males of the species have been known to live up to 23 years, while females can live up to 21 years.

Although not yet endangered, the far eastern fire salamander population is in decline. Primary threats include damming of rivers (which disrupts the salamander's breeding) and pollution. They are also threatened by the recent introduction of predatory fish, such as the mosquitofish. These fish were intended to control the mosquito population, but they also feed on young salamanders.

Many of the animals on O'Reilly covers are endangered; all of them are important to the world. To learn more about how you can help, go to *animals.oreilly.com*.

The cover image is from *Wood's Illustrated Natural History*. The cover fonts are URW Typewriter and Guardian Sans. The text font is Adobe Minion Pro; the heading font is Adobe Myriad Condensed; and the code font is Dalton Maag's Ubuntu Mono.